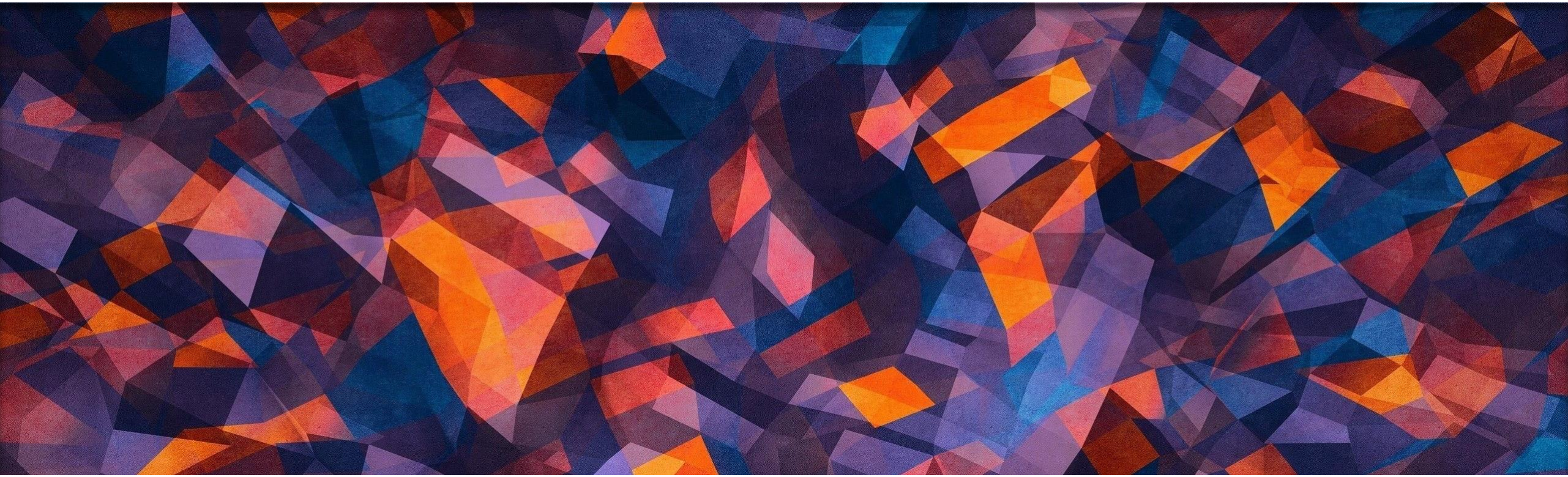
Universität St.Gallen



Event-driven and Process-oriented Architectures

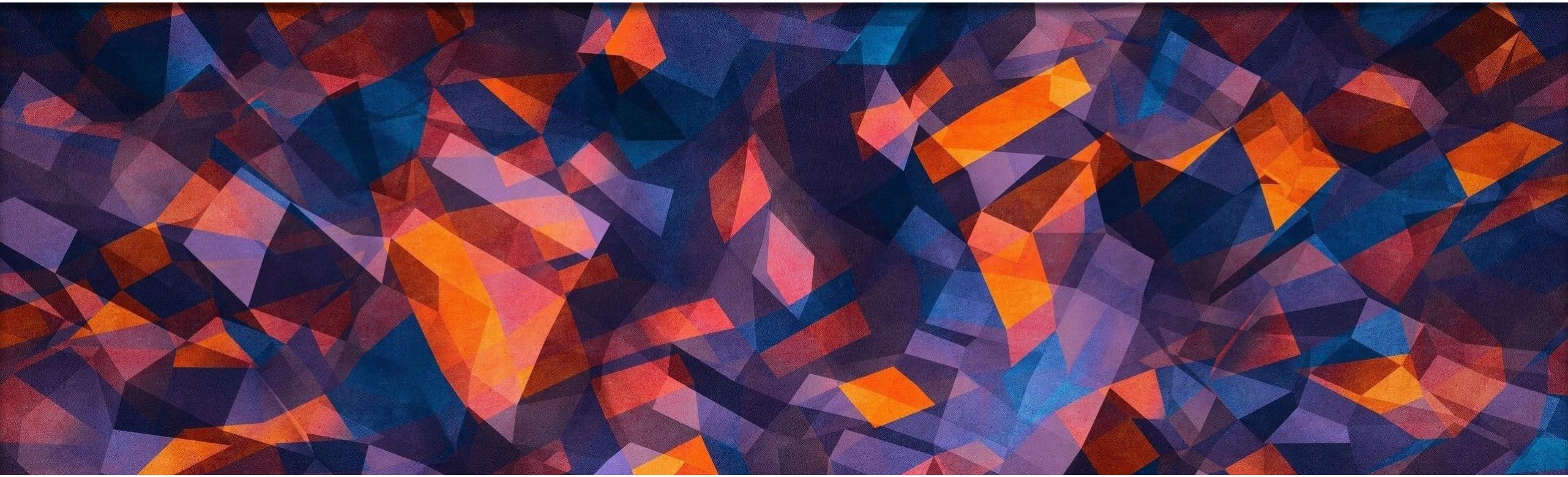
Process Orchestration and Process Automation: Part 1

Lecture 3

Agenda

- Motivating Example: Flowing Retail Example (Continued)
- Coordination: Orchestration
- Process Engines: Fundamentals of Process Automation

Universität St.Gallen



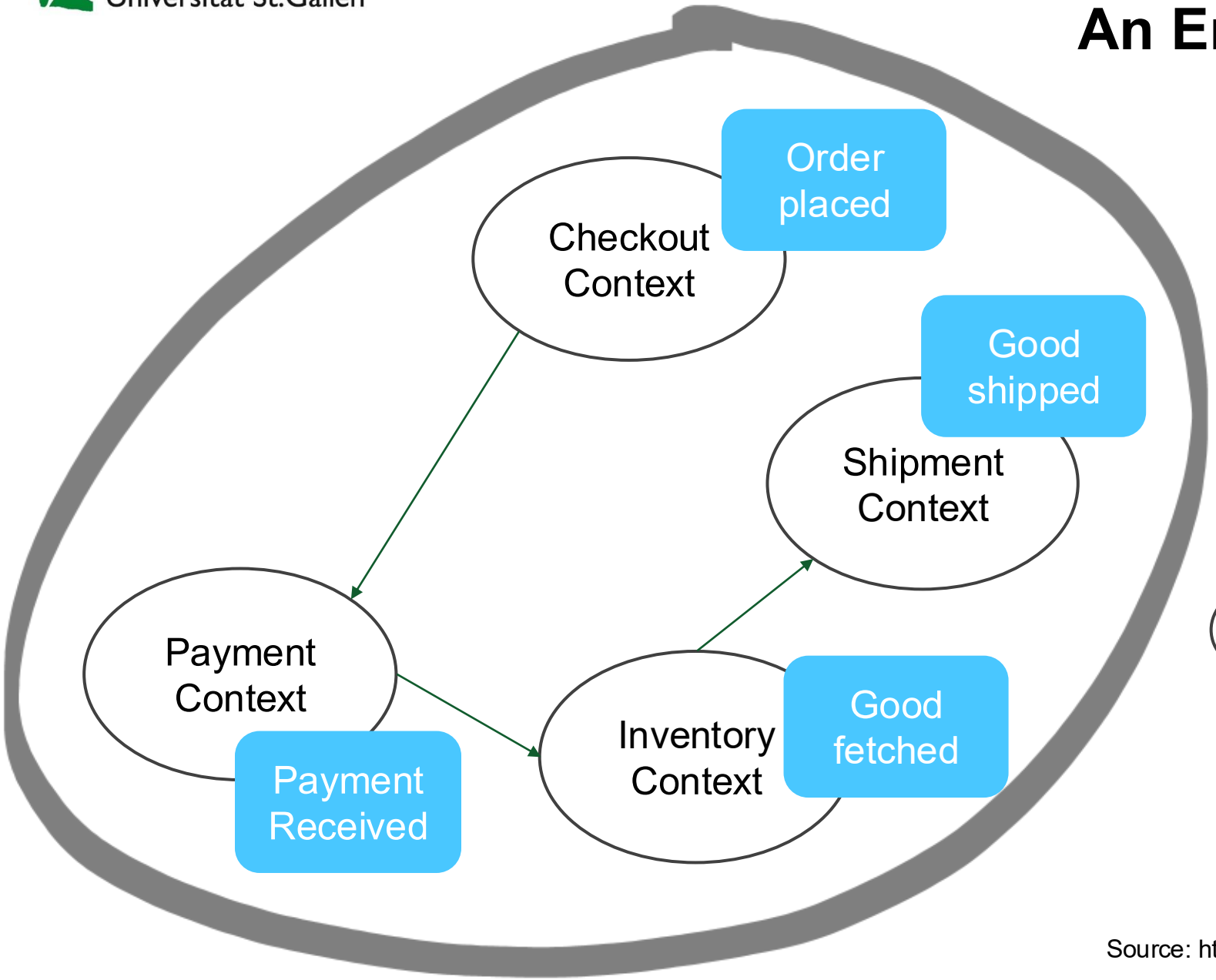
Event-driven and Process-oriented Architectures

Motivating Example

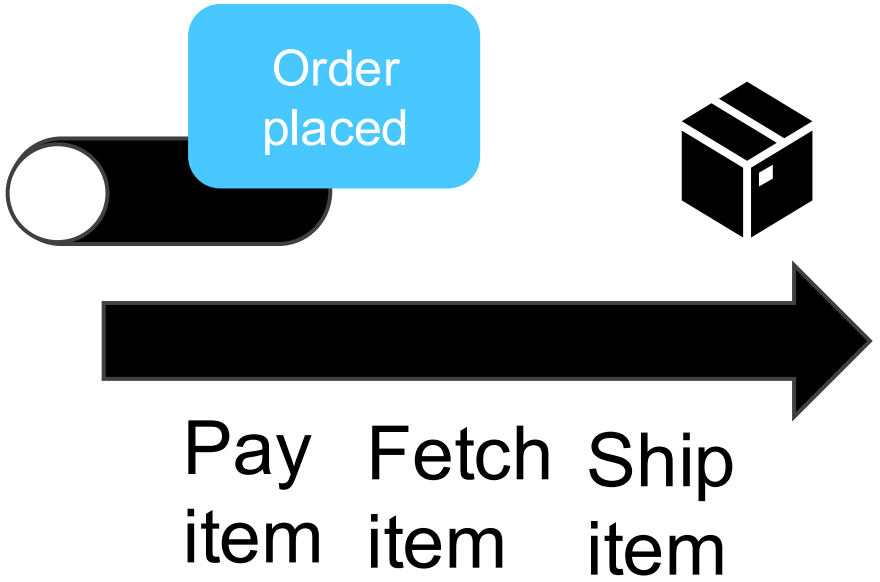
Lecture 3

In distributed systems, choreography is the superior pattern. It maximizes decoupling. Therefore, it should be our default architectural choice.

Order Fulfillment Flow
An End-to-end Process

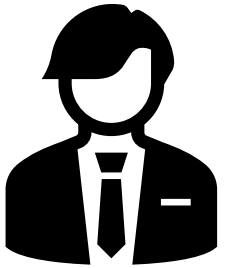


- When the button is pressed an order is placed and fulfilled



Source: https://www.infoq.com/articles/events-workflow-automation/

Choreography works here.



However:

– The workflow lacks visibility

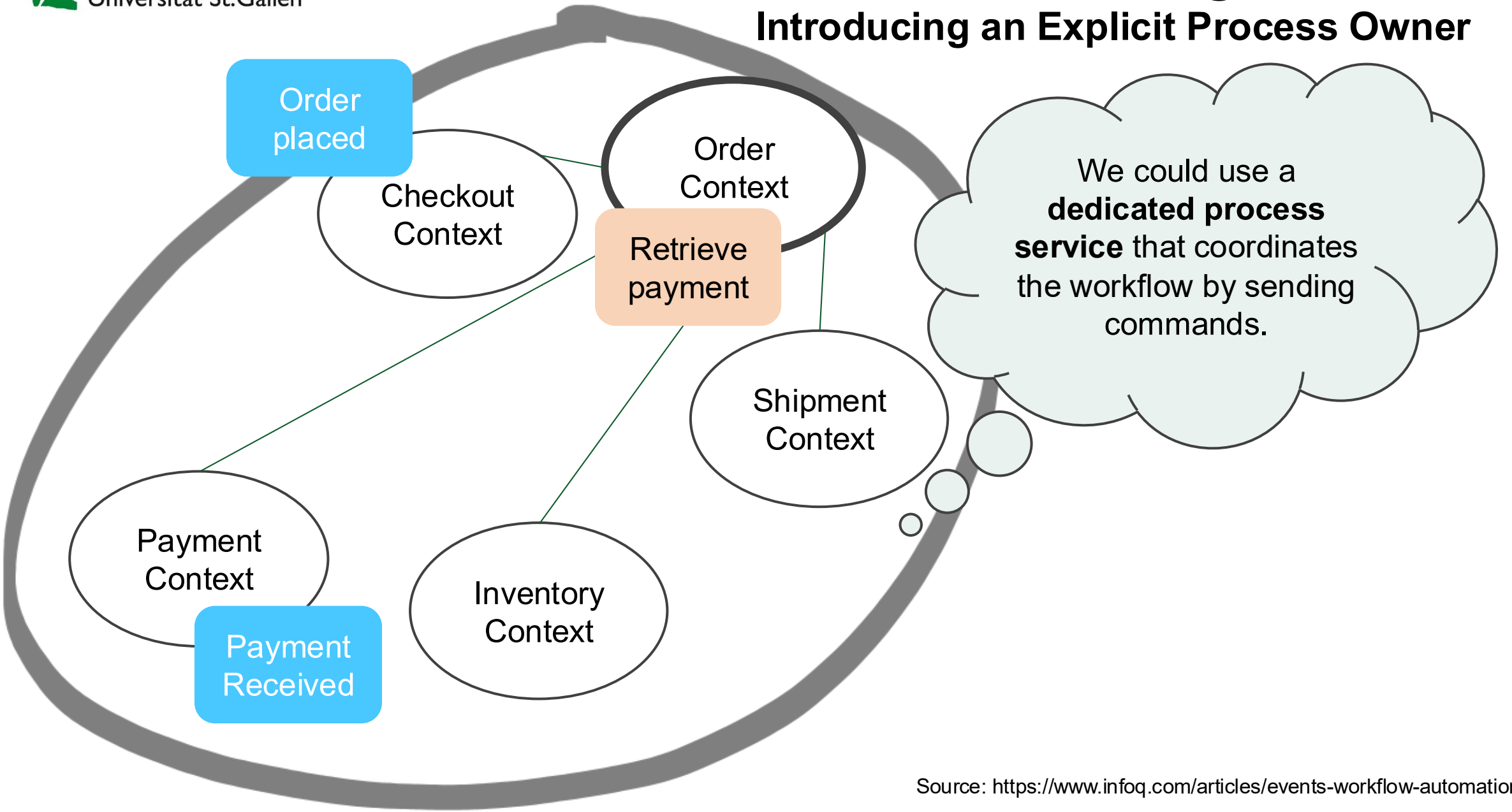
– No single service owns the end-to-end process

– Changes to the flow are difficult and distributed

Should we consider orchestration instead?

Orchestrating the Flow

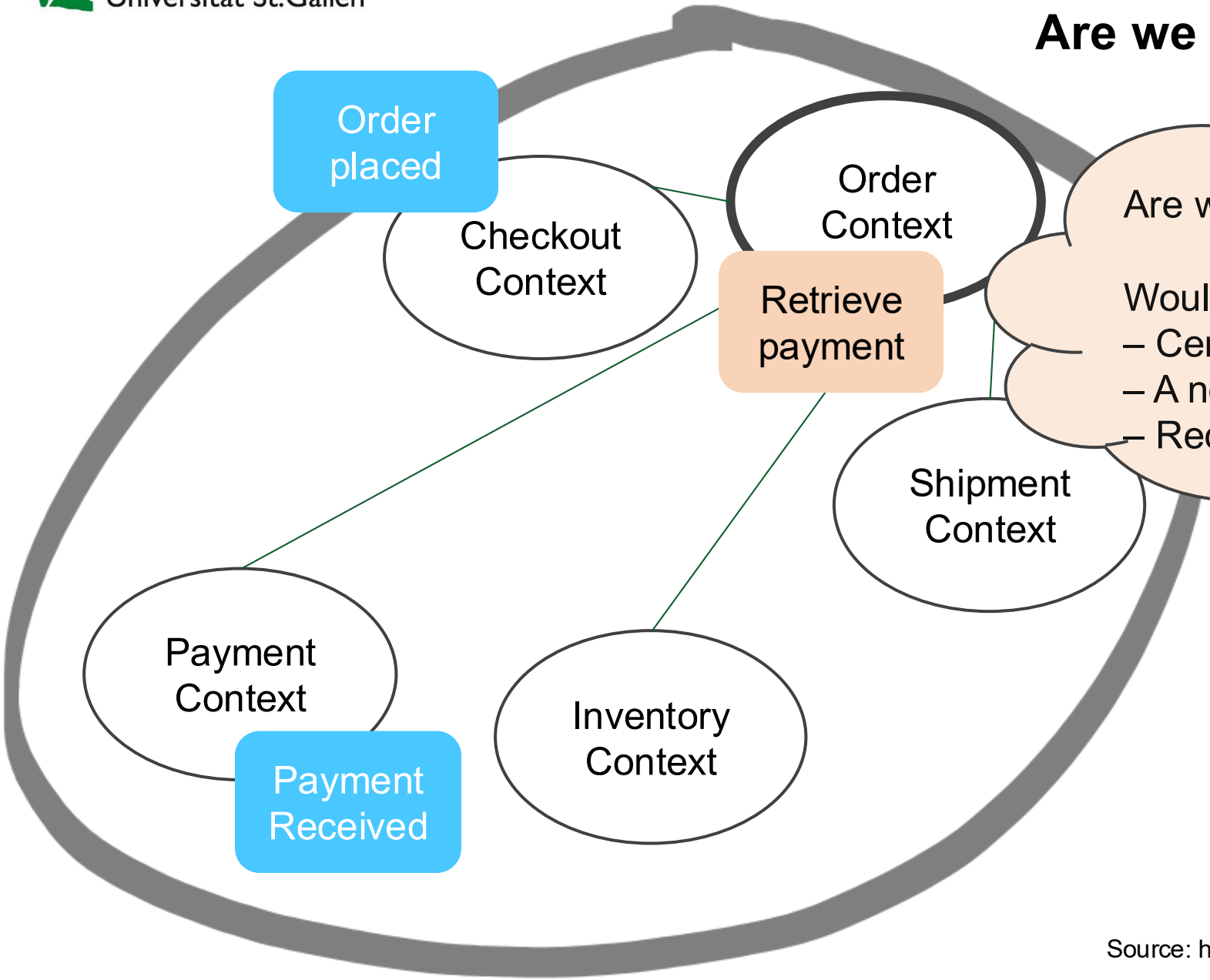
Introducing an Explicit Process Owner



Source: https://www.infoq.com/articles/events-workflow-automation/

Orchestrating the Flow

Are we Reintroducing the ESB?

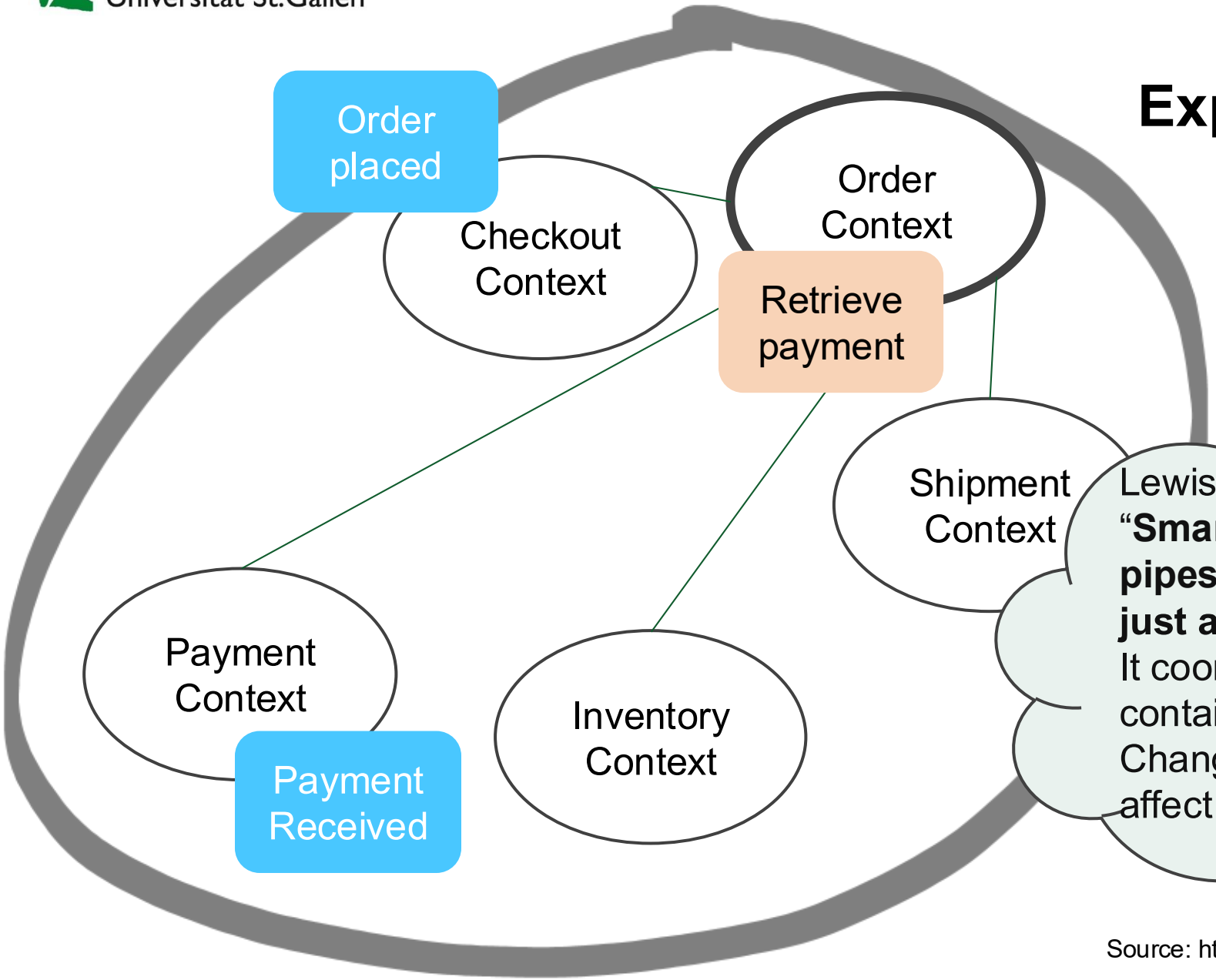


Are we rebuilding an ESB?

Would orchestration mean:
– Centralized control?
– A new point of failure?
– Reduced team autonomy?

Source: https://www.infoq.com/articles/events-workflow-automation/

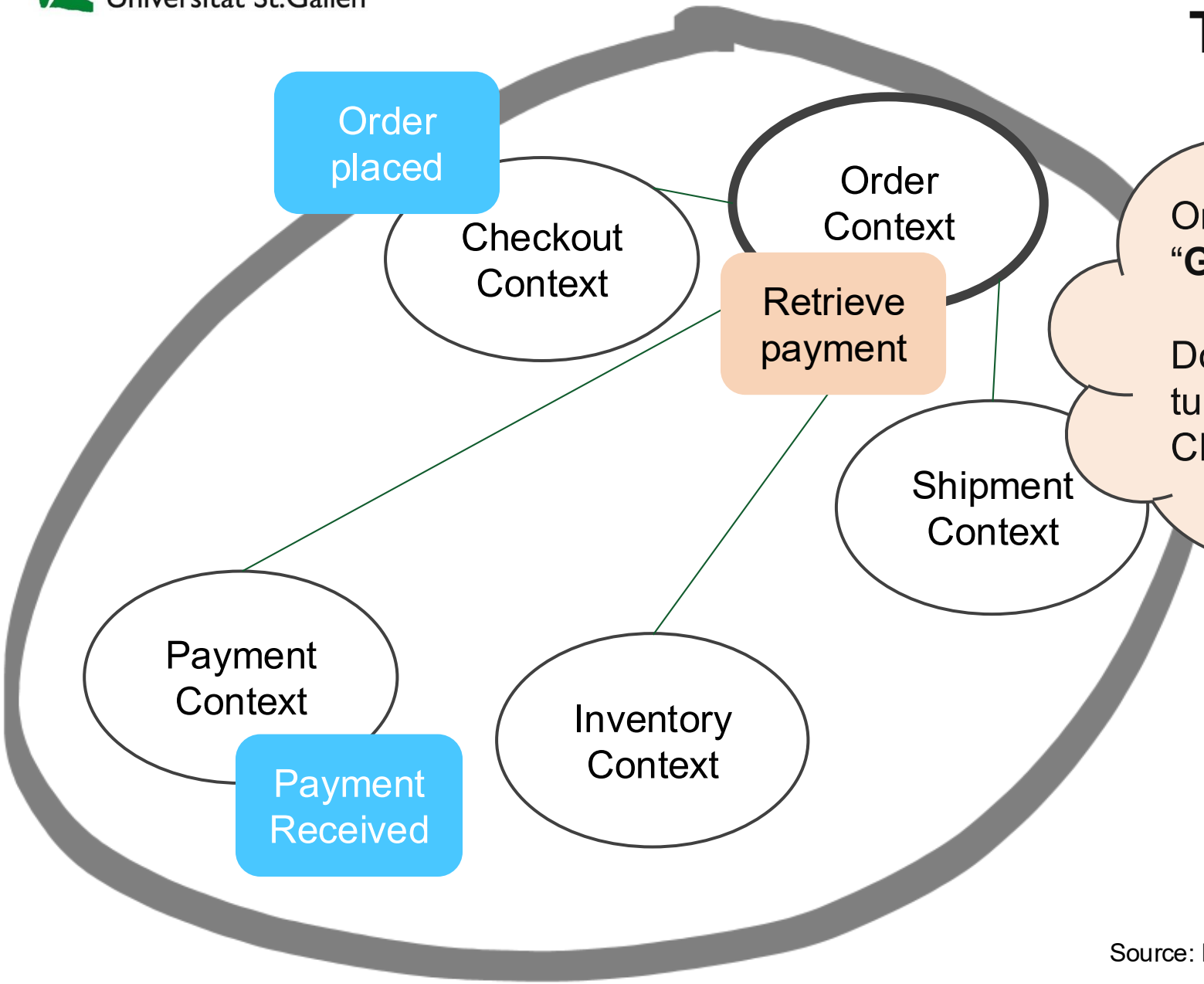
Orchestrating the Flow
Orchestrator: Explicit Process Owner



Lewis & Fowler advocate "**Smart endpoints, dumb pipes.**" The **orchestrator** is **just another service.**
It coordinates — it does not contain domain logic.
Changes to the workflow affect only this service.

Source: https://www.infoq.com/articles/events-workflow-automation/

Orchestrating the Flow
The God Service Risk

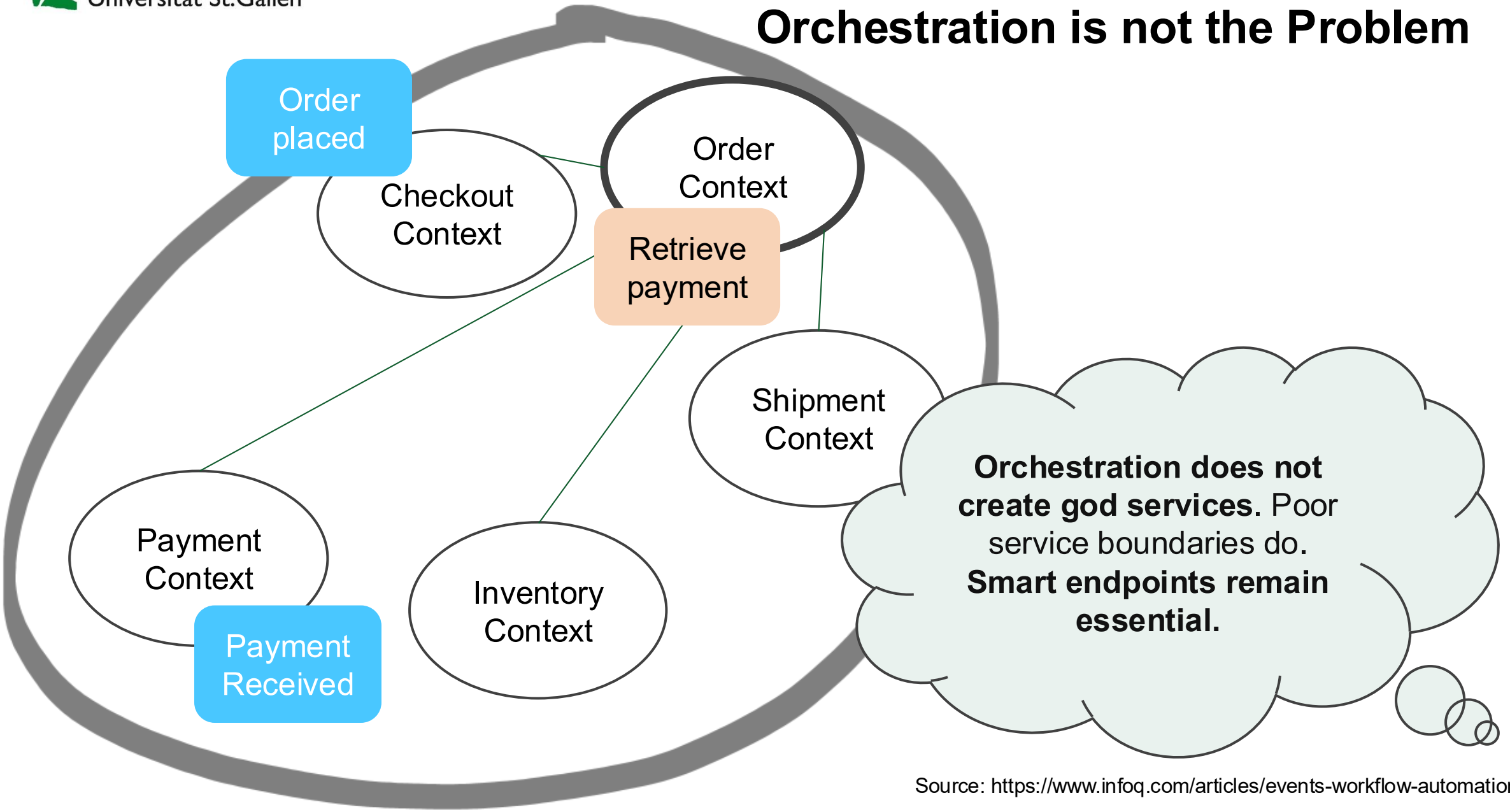


Orchestration and the "**God Service**" Risk

Does central process logic turn other services into CRUD enpoints?

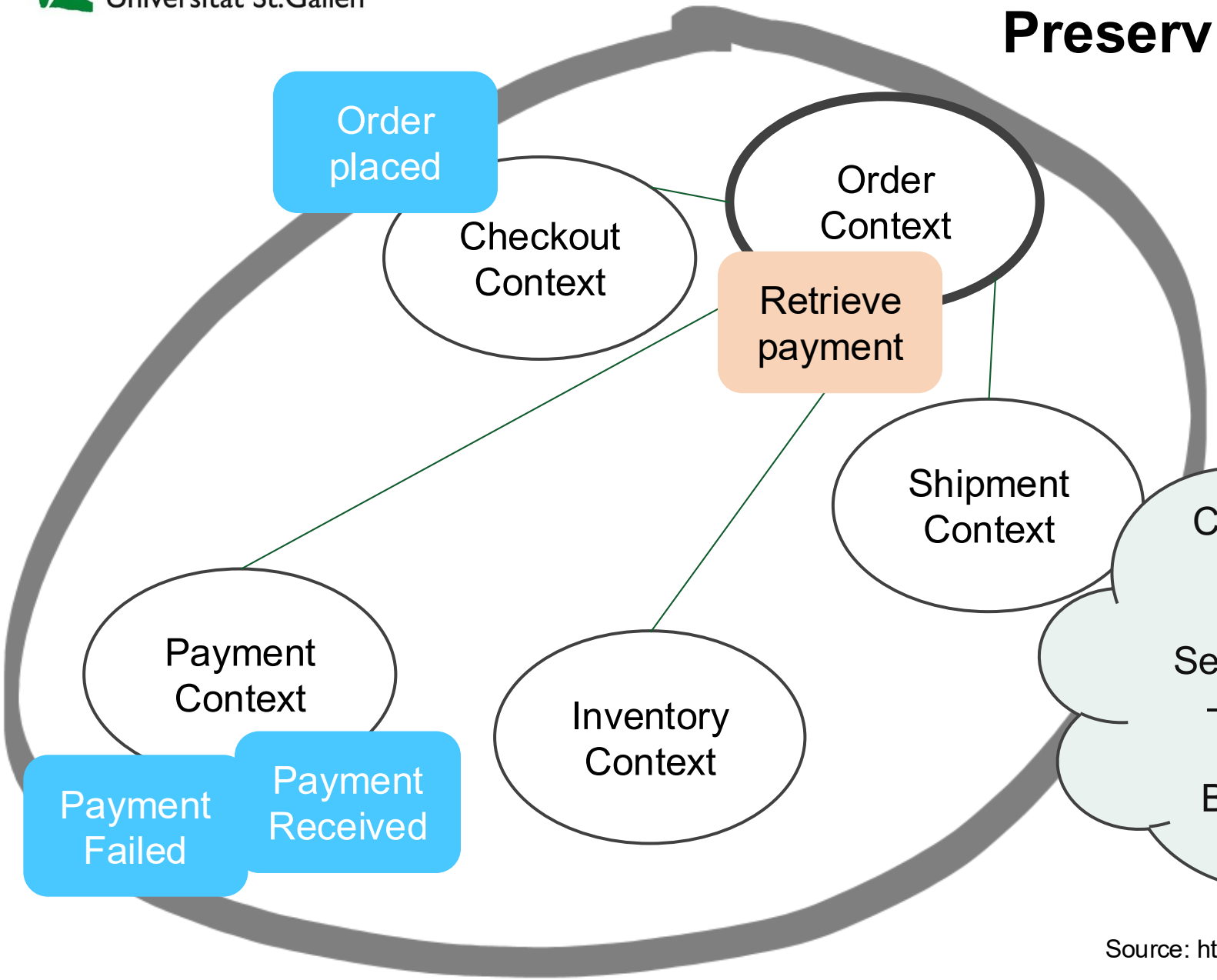
Source: https://www.infoq.com/articles/events-workflow-automation/

Orchestrating the Flow
Orchestration is not the Problem



Source: https://www.infoq.com/articles/events-workflow-automation/

Orchestrating the Flow
Preserving Smart Endpoints



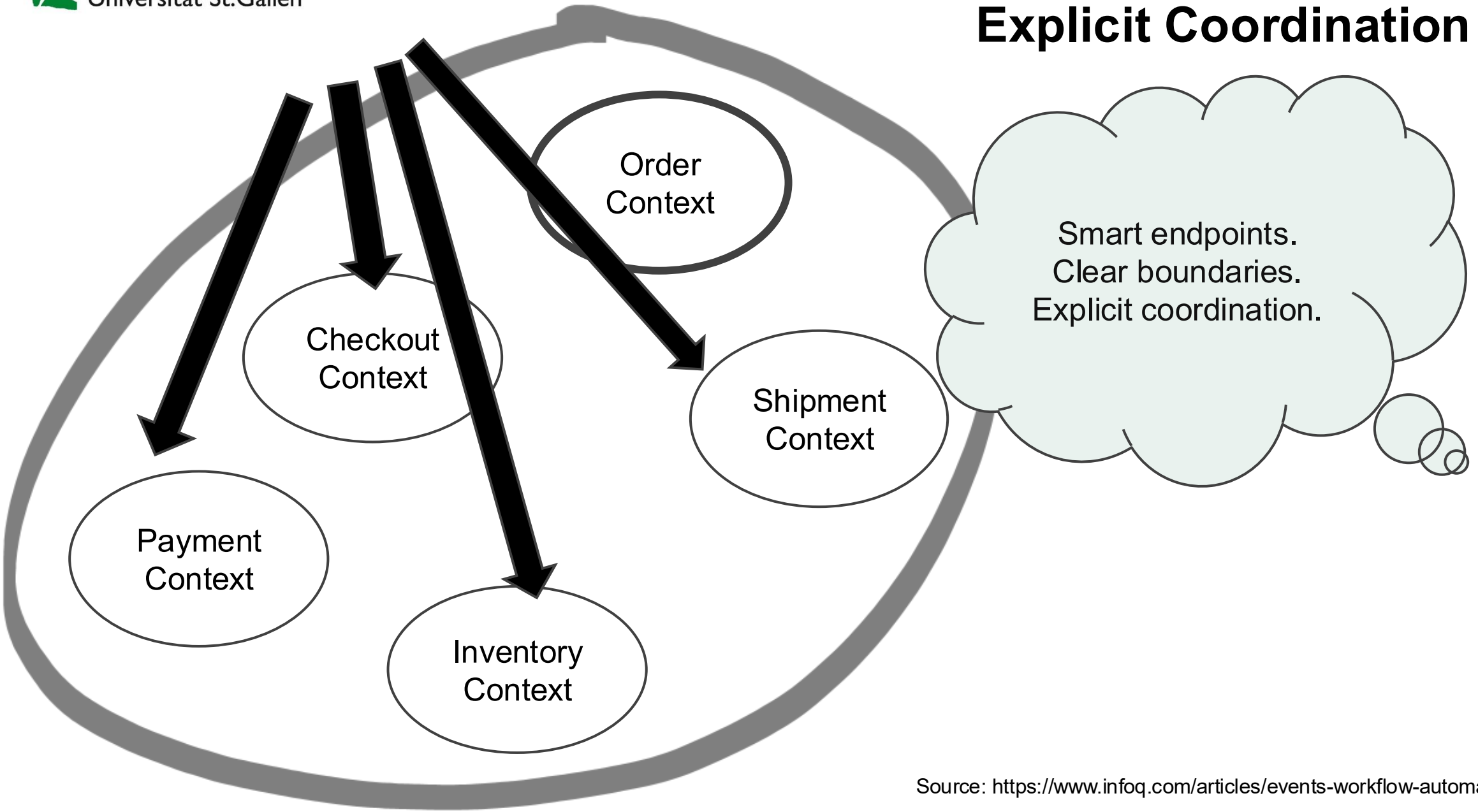
Commands trigger domain behavior.

Services emit domain events — not CRUD responses.

Boundaries remain intact.

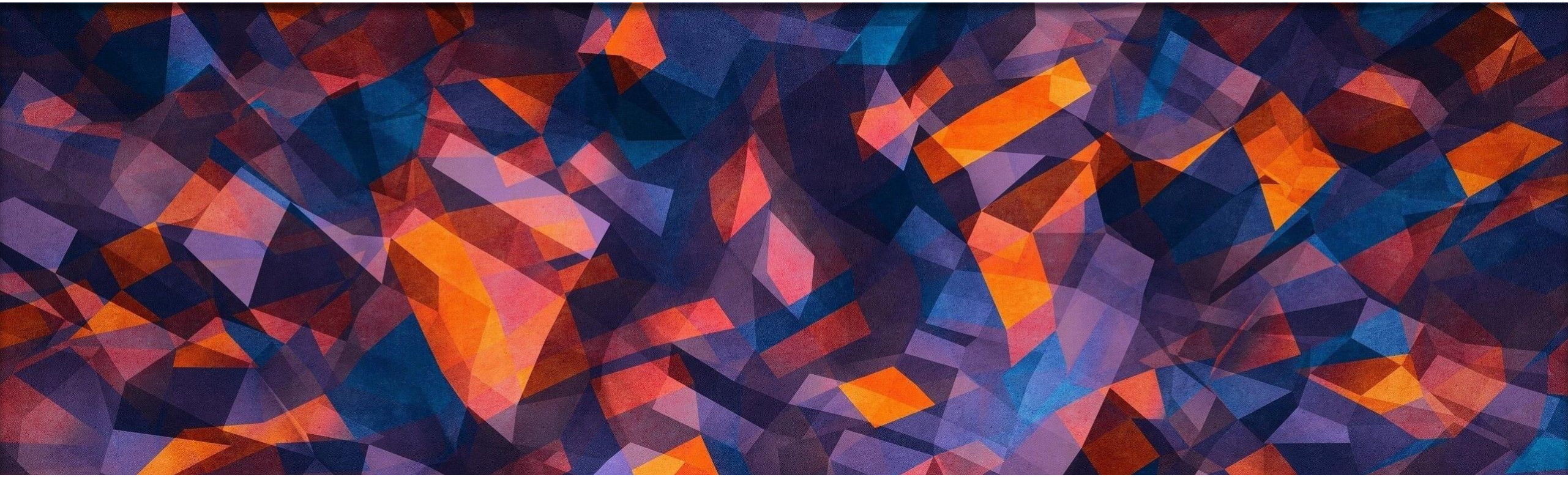
Source: https://www.infoq.com/articles/events-workflow-automation/

Smart Endpoints, Explicit Coordination



Source: https://www.infoq.com/articles/events-workflow-automation/

Universität St.Gallen

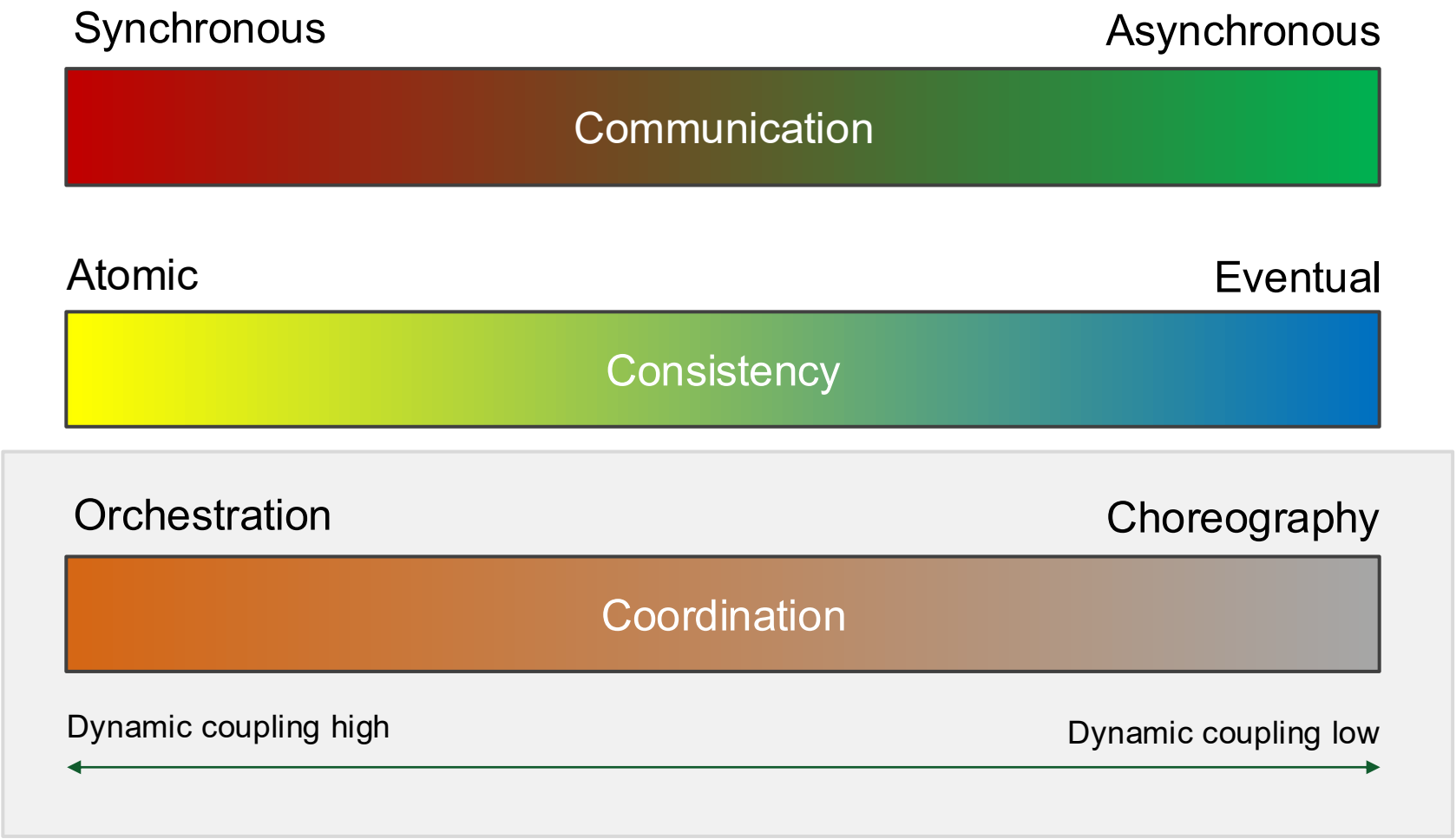


Event-driven and Process-oriented Architectures

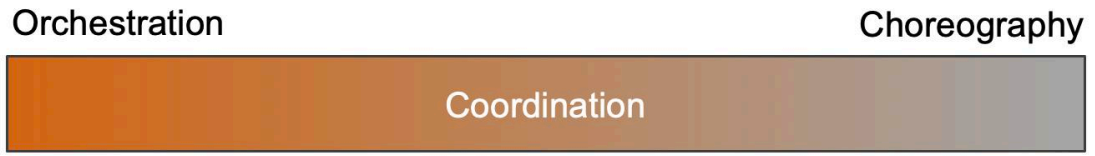
Coordination: Orchestration

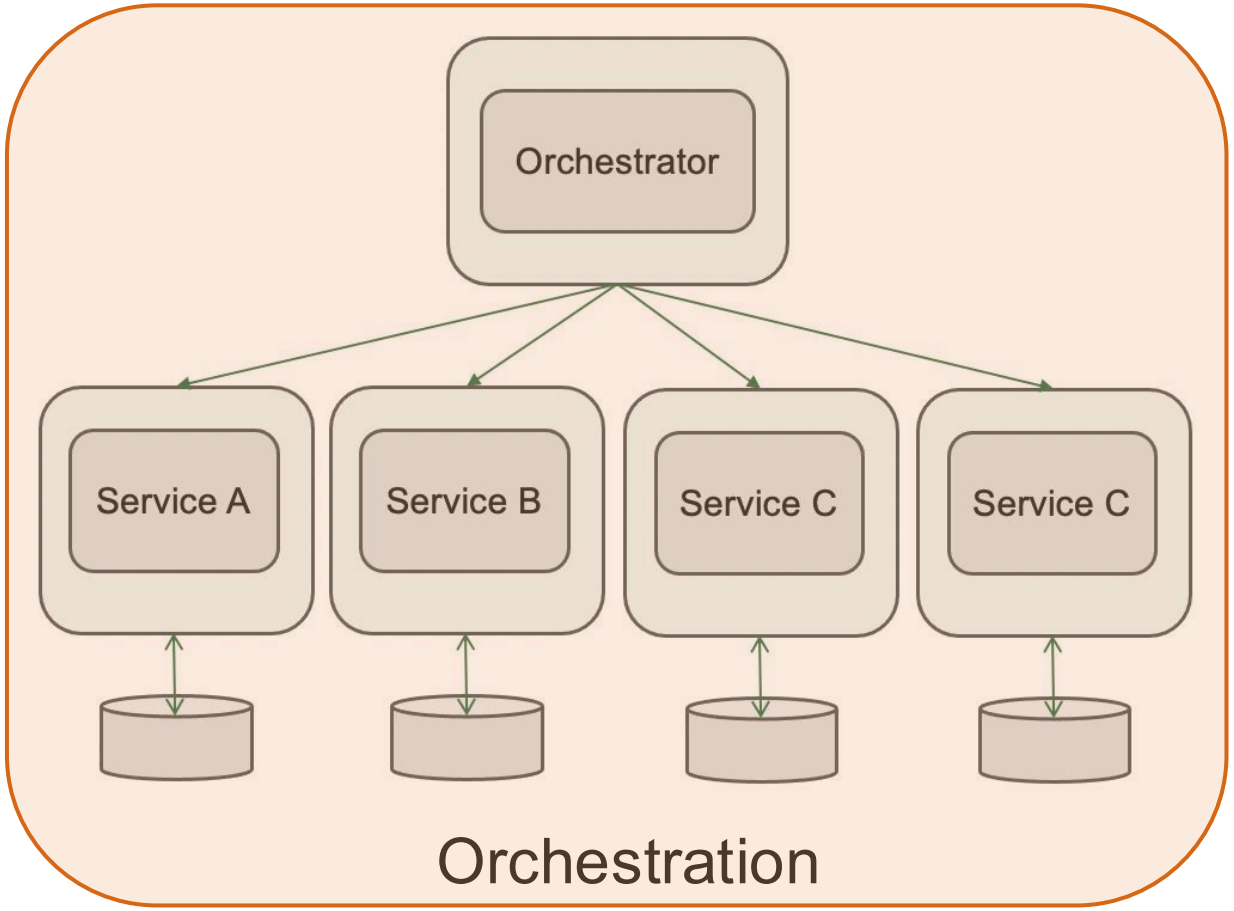
Lecture 3

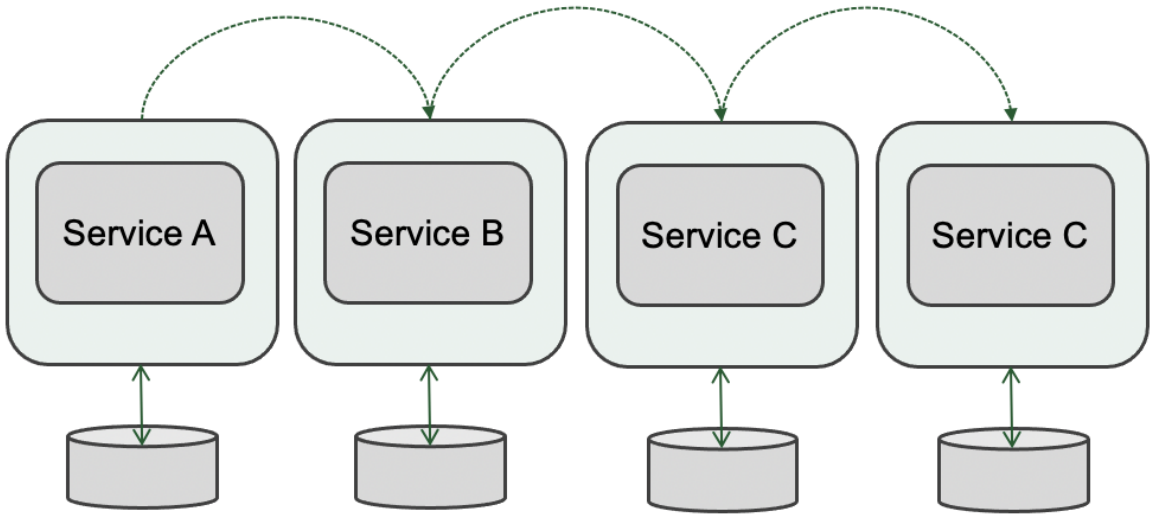
Dynamic Quantum Coupling (Recap)
Three Coupling Forces



Orchestration versus Choreography
Two Fundamental Coordination Patterns

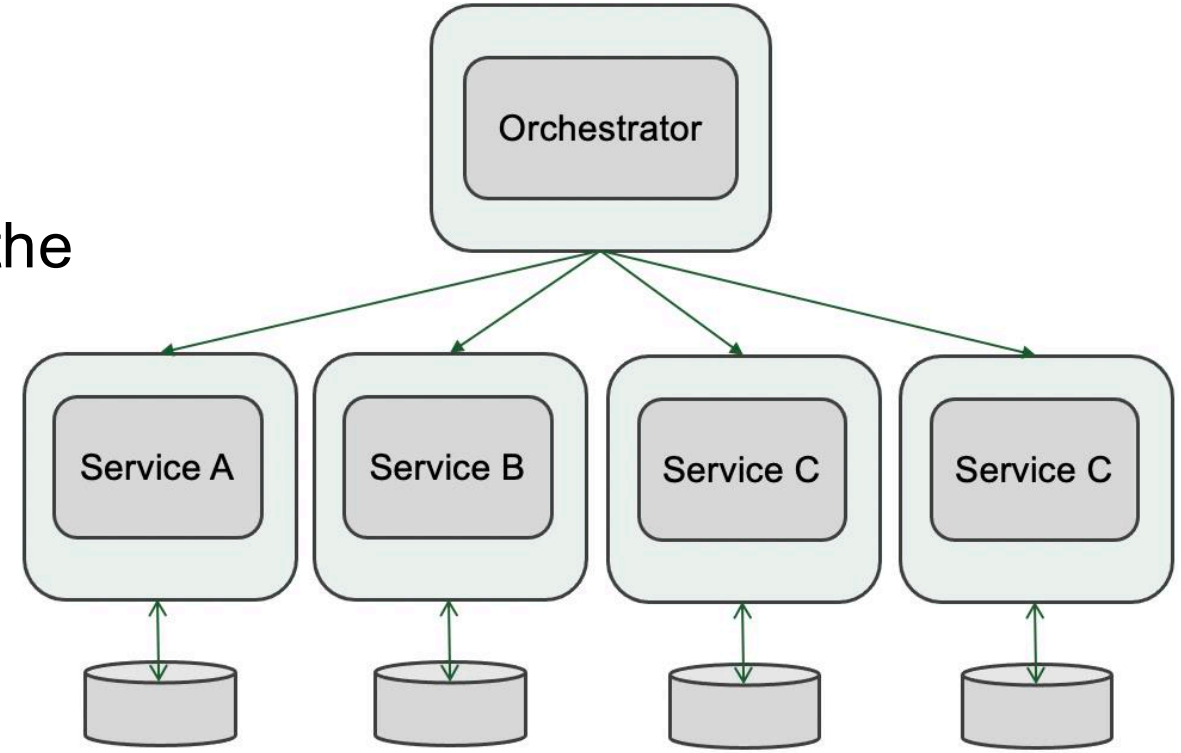






Orchestration Pattern

- Uses an orchestrator (mediator) to:
  - Manage workflow state
  - Coordinate steps
  - Handle errors and alternative paths
- Contains no domain logic outside the workflow it coordinates
- One orchestrator per workflow and no global orchestrator

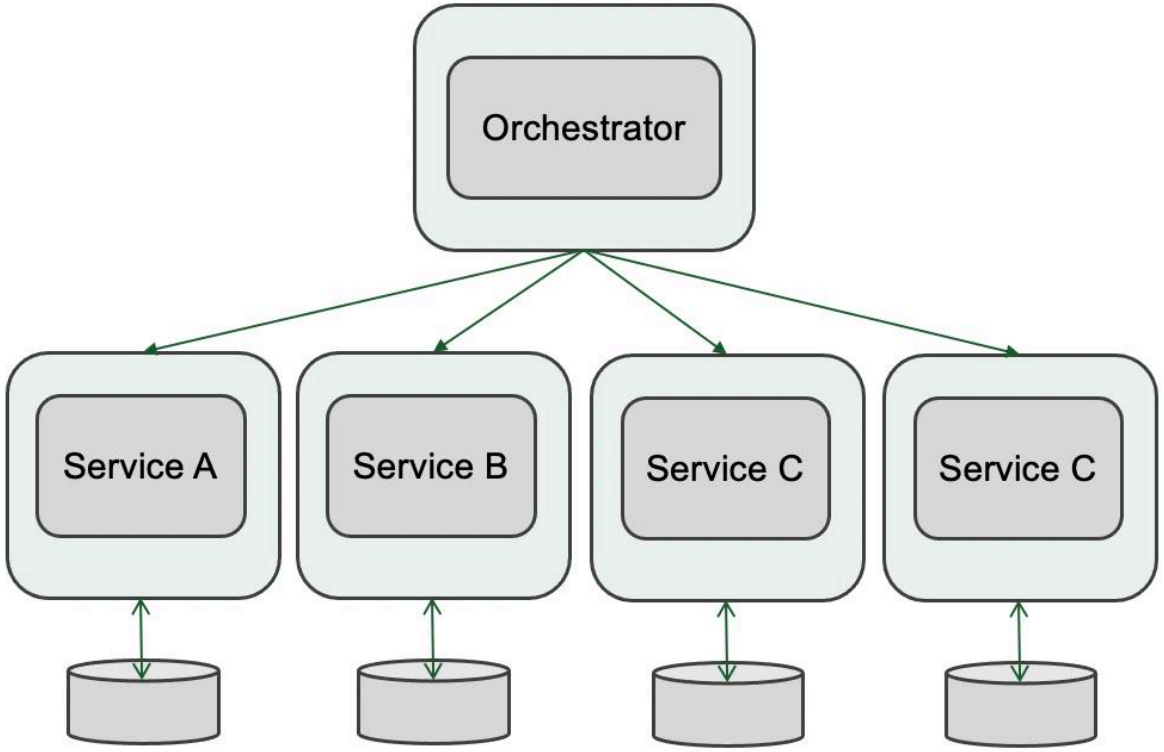


Orchestration Pattern

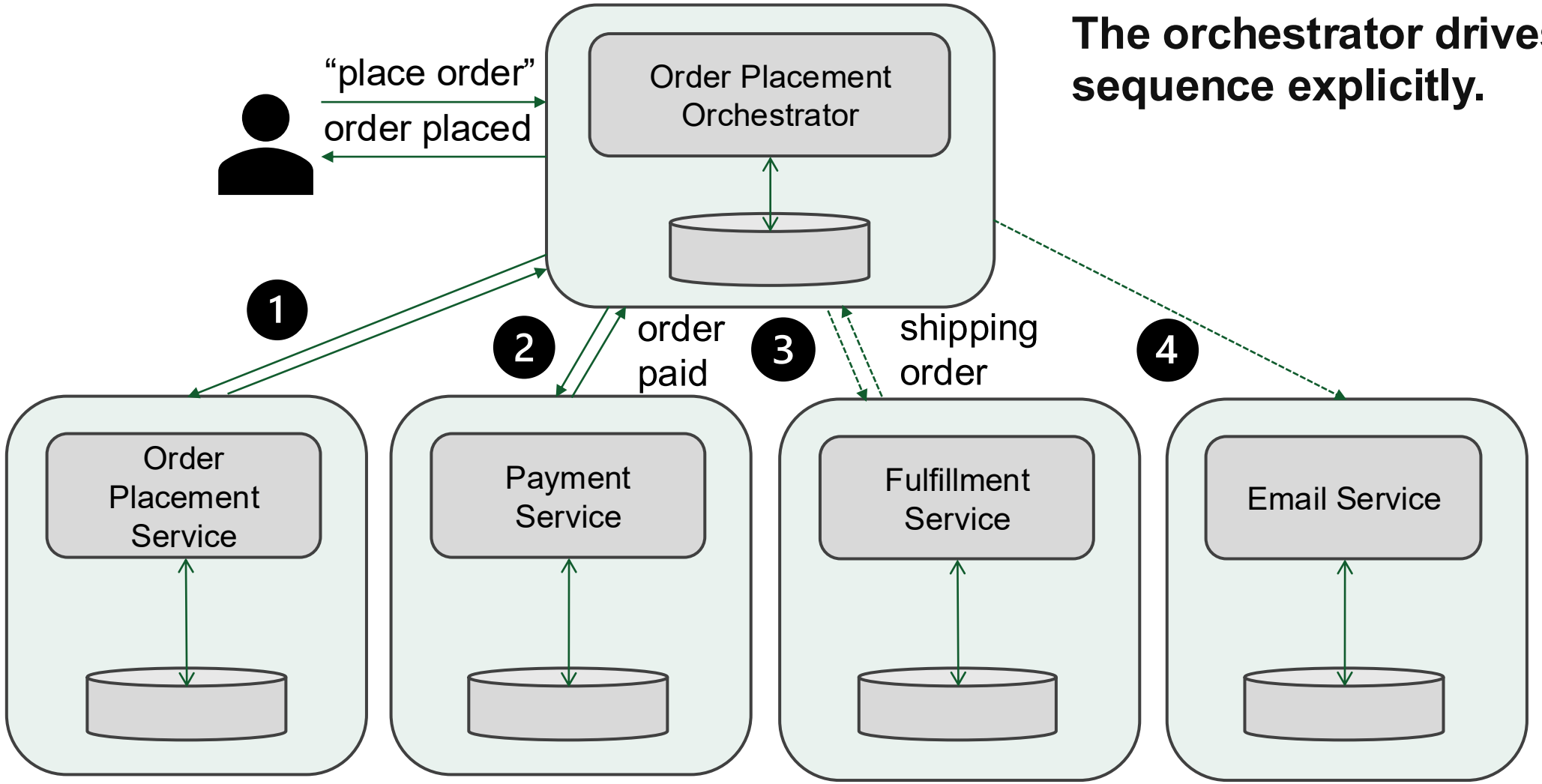
When to use

When workflow logic includes:

- Multiple execution paths
- Explicit error handling
- Compensations

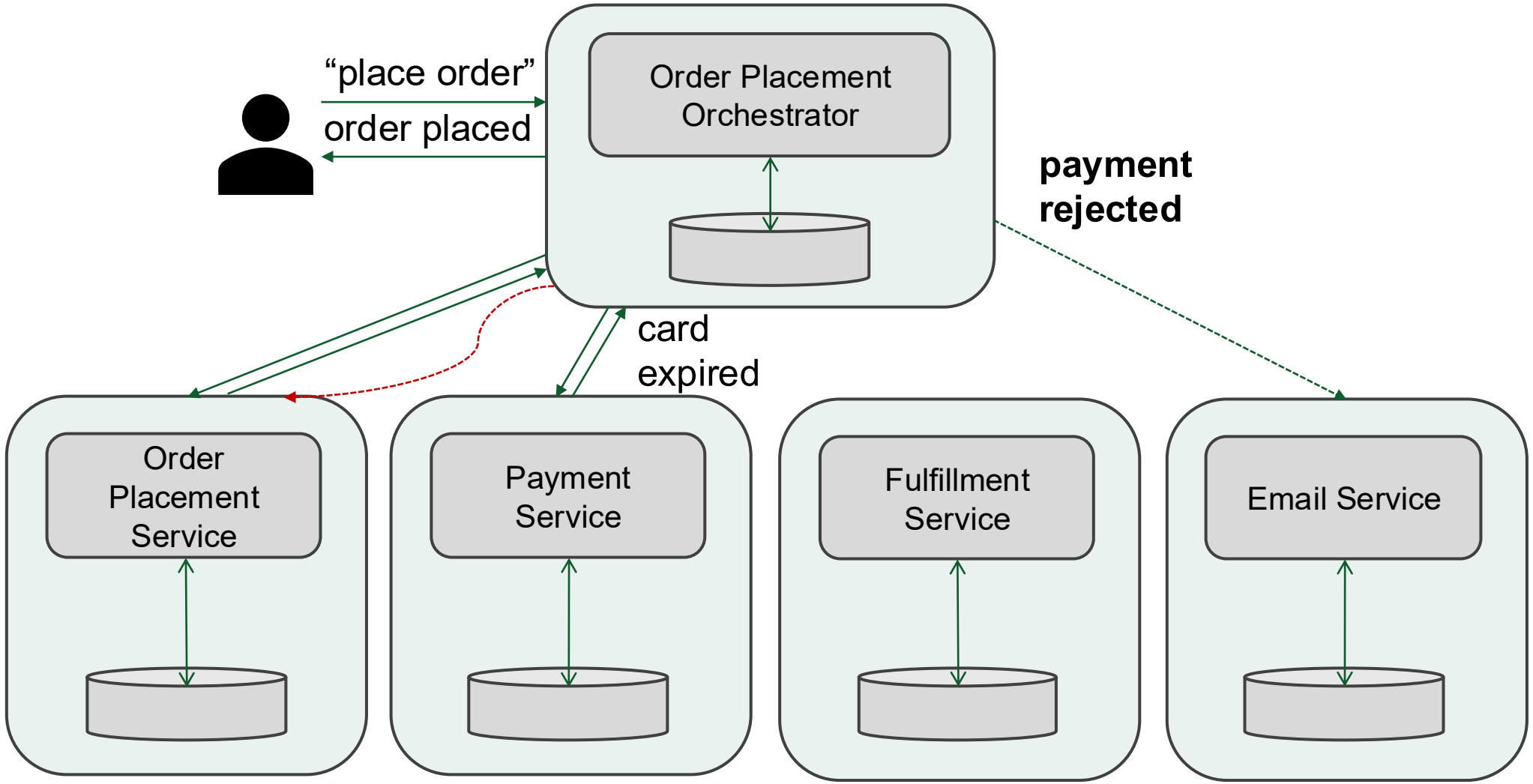


Happy Path Using an Orchestrator



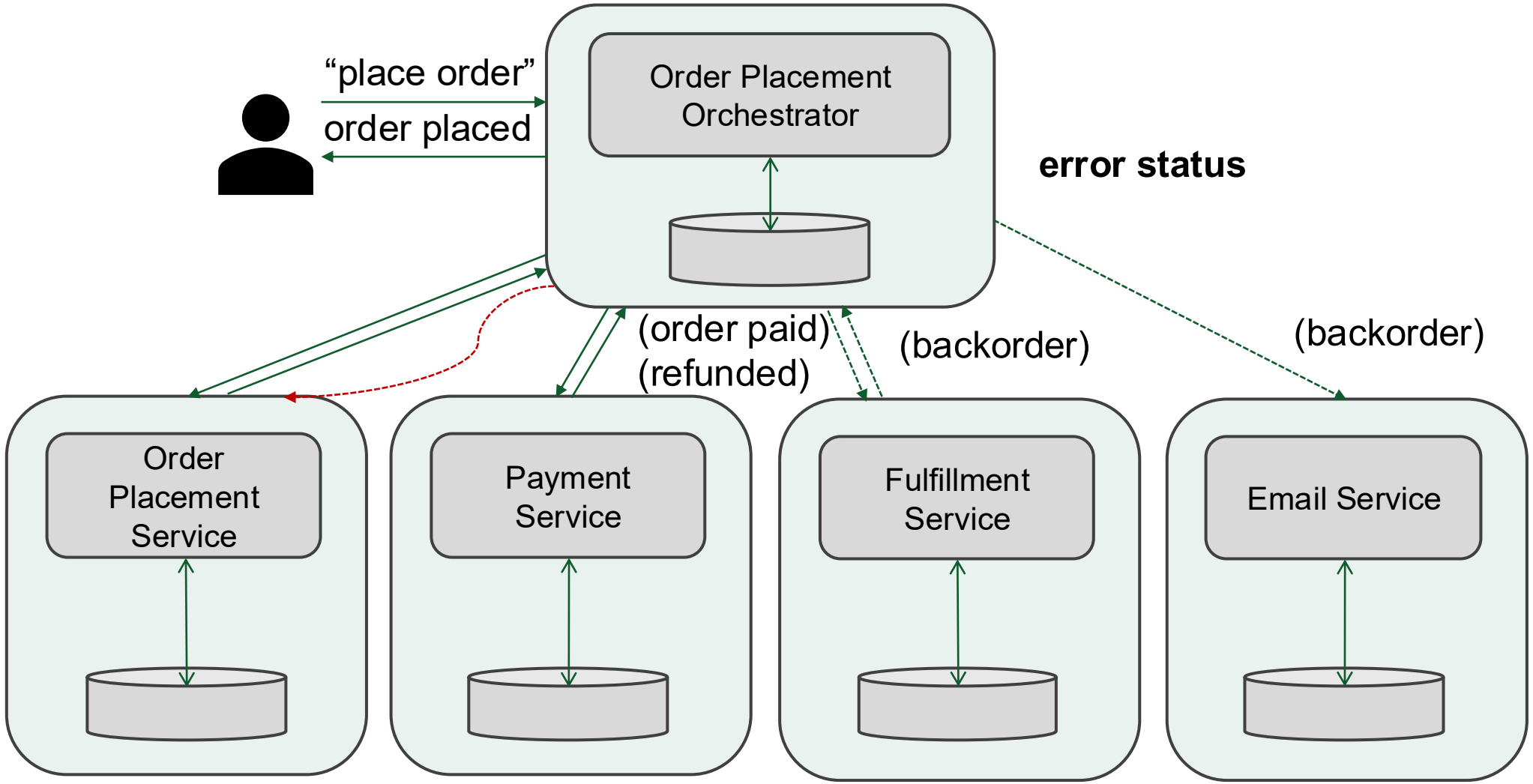
The orchestrator drives the sequence explicitly.

Payment Rejection Error



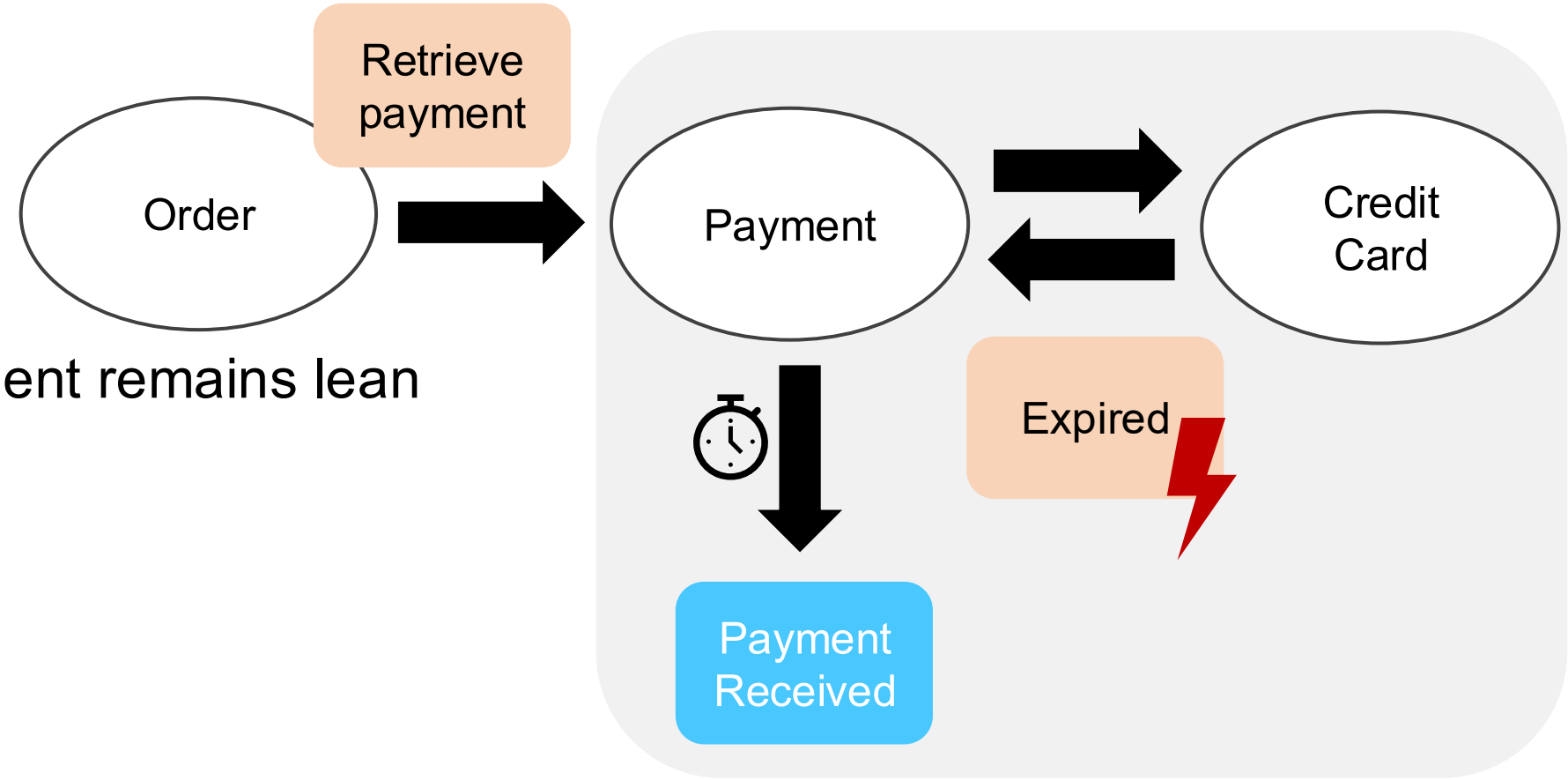
Saga Pattern: each service maintains its own transactional state

Handling Back Orders



Saga Pattern: each service maintains its own transactional state

Long-Running Services Require State



The client remains lean

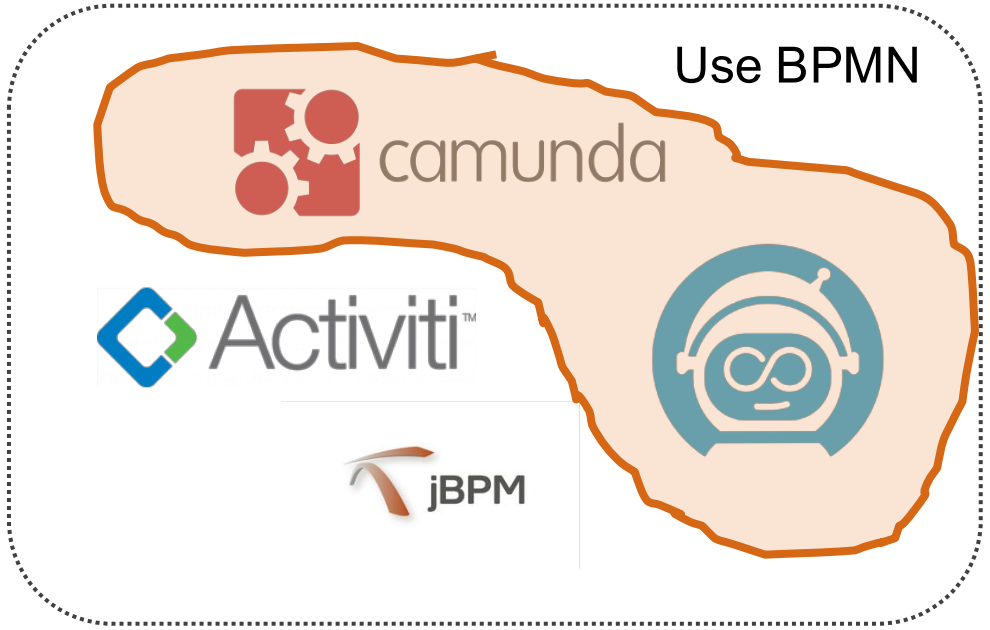
Long-running services require persistent state.
Otherwise, complexity leaks to the client.

Source: https://www.infoq.com/articles/events-workflow-automation/

Implementing Persistent State

- You must make a design decision:
  - Build your own durable workflow state
  - Use a workflow / state-machine engine

BPM engines (process centric)



Code-first orchestrator

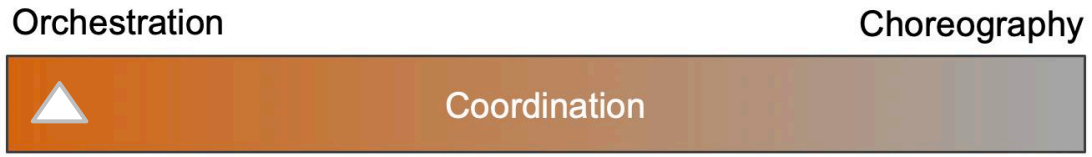
Cadence

AWS Step Functions

conductor

Source: https://www.infoq.com/articles/events-workflow-automation/

Trade-offs Workflow-based Orchestration



Pros	Cons
Explicit workflow ownership	Central coordination point
Structured error handling	Requires durable state
Recoverability	Potential scaling hotspot
Clear state management	Increased coordination coupling

Sweetspot for Orchestration

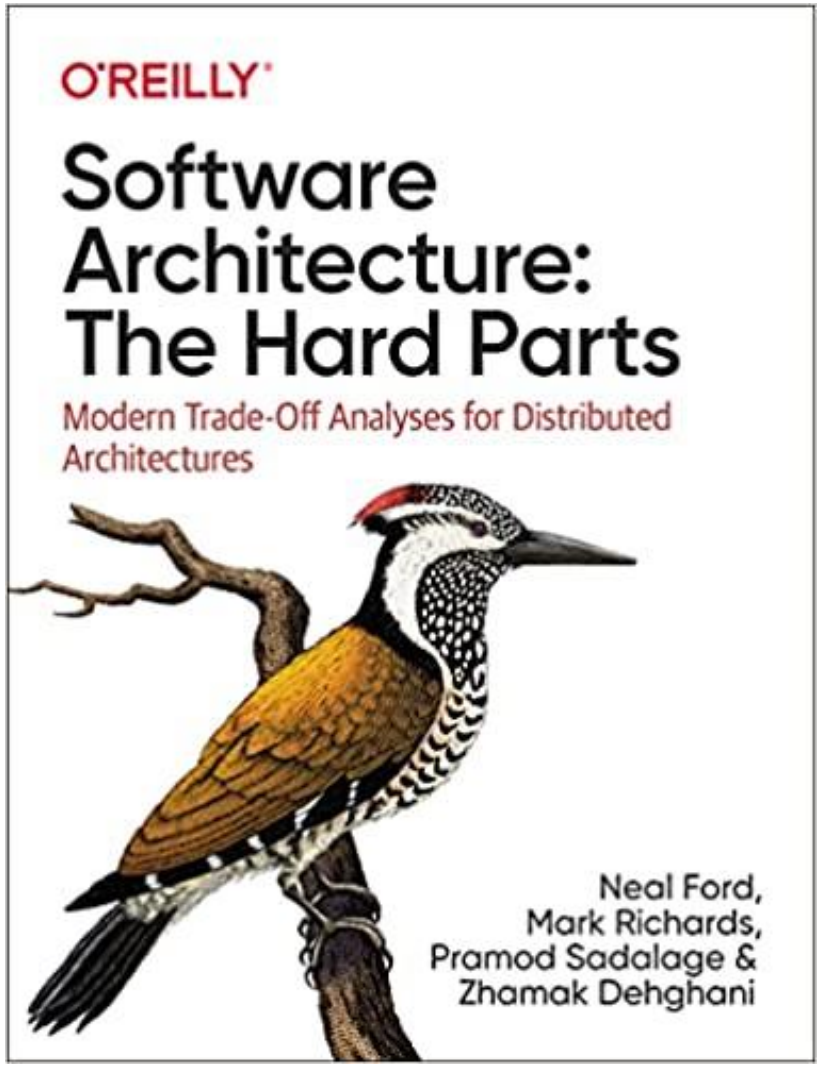
- The more business rules span services, the more useful an orchestrator becomes
- Best suited for workflows with multiple paths and explicit error handling
- Particularly effective for strongly coordinated sagas

- Used by the following dynamic coupling patterns

Epic Saga	Synchronous	Atomic	Very high
Epic Saga	Synchronous	Atomic	Very high
Fairy Tale Saga	Synchronous	Eventual	High
Fairy Tale Saga	Synchronous	Eventual	High
Parallel Saga	Asynchronous	Eventual	Low

Universität St.Gallen

References and Further Material



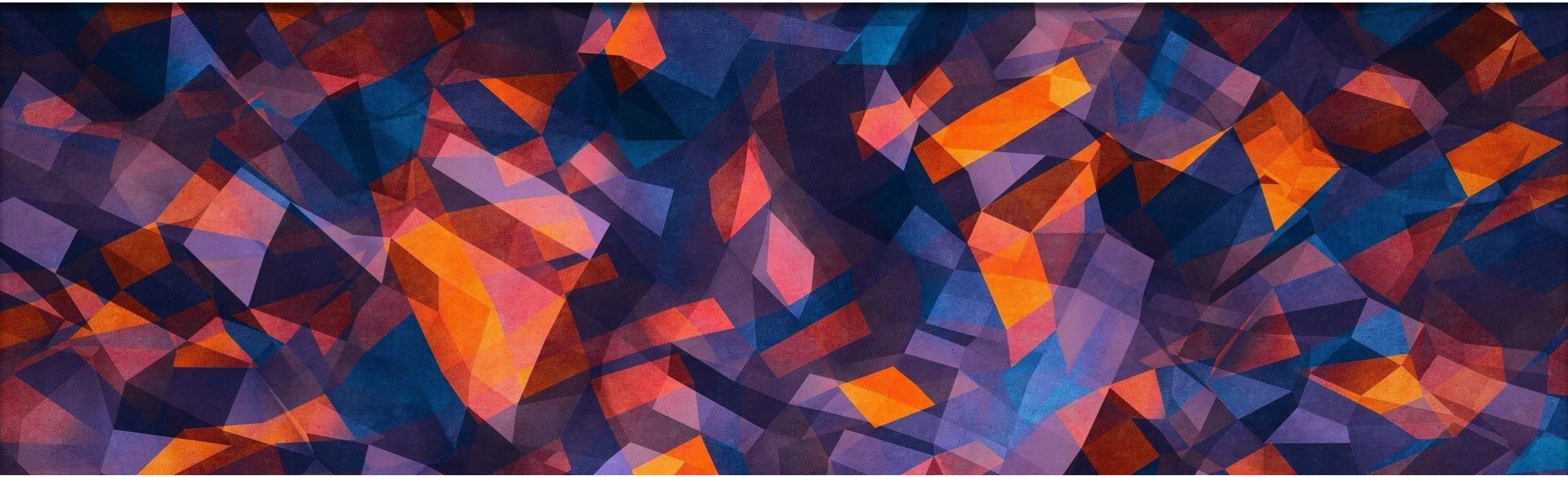
Based on Chapter 11 – Managing Distributed Workflows

Further References

The Role of Workflow in Microservices

https://www.youtube.com/watch?v=UIepHI_JPwE

Universität St.Gallen



Event-driven and Process-oriented Architectures

Fundamentals of Process Automation with Process Engines

Lecture 3

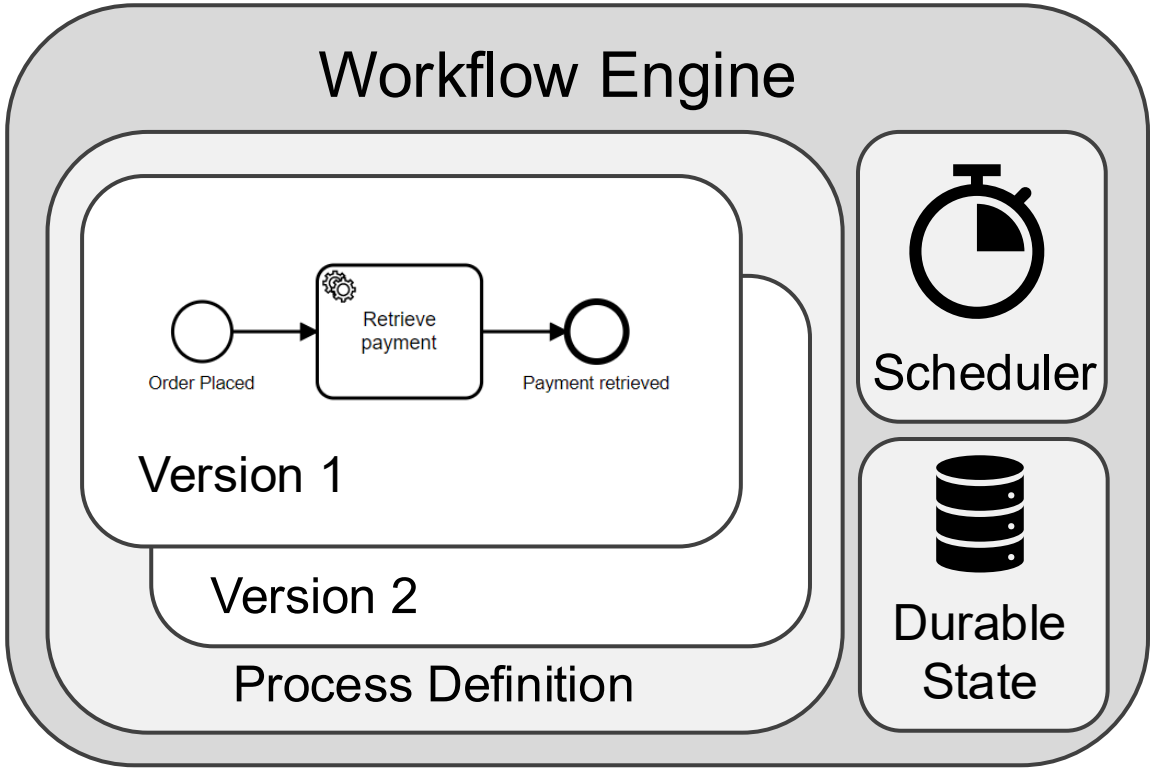
Agenda

- Workflow Engines: When and Why?
- Development of Process Solutions
- Orchestrating Everything

Agenda

- **Workflow Engines: When and Why?**
- Development of Process Solutions
- Orchestrating Everything

What a Workflow Engine Actually Provides



Durable Workflow State

- Tracks all running instances
- Persists current state and history

Built-in Time & Retry Handling

- Timers, deadlines, escalations
- Automatic retry logic

Process Versioning

- Multiple versions run in parallel
- Running instances survive deployments

Architectural Consequences of Using an Engine

Durable Workflow State

→ Workflow state is externalized

→ Long-running processes become operationally manageable

Built-in Time & Retry Handling

→ Less custom infrastructure code

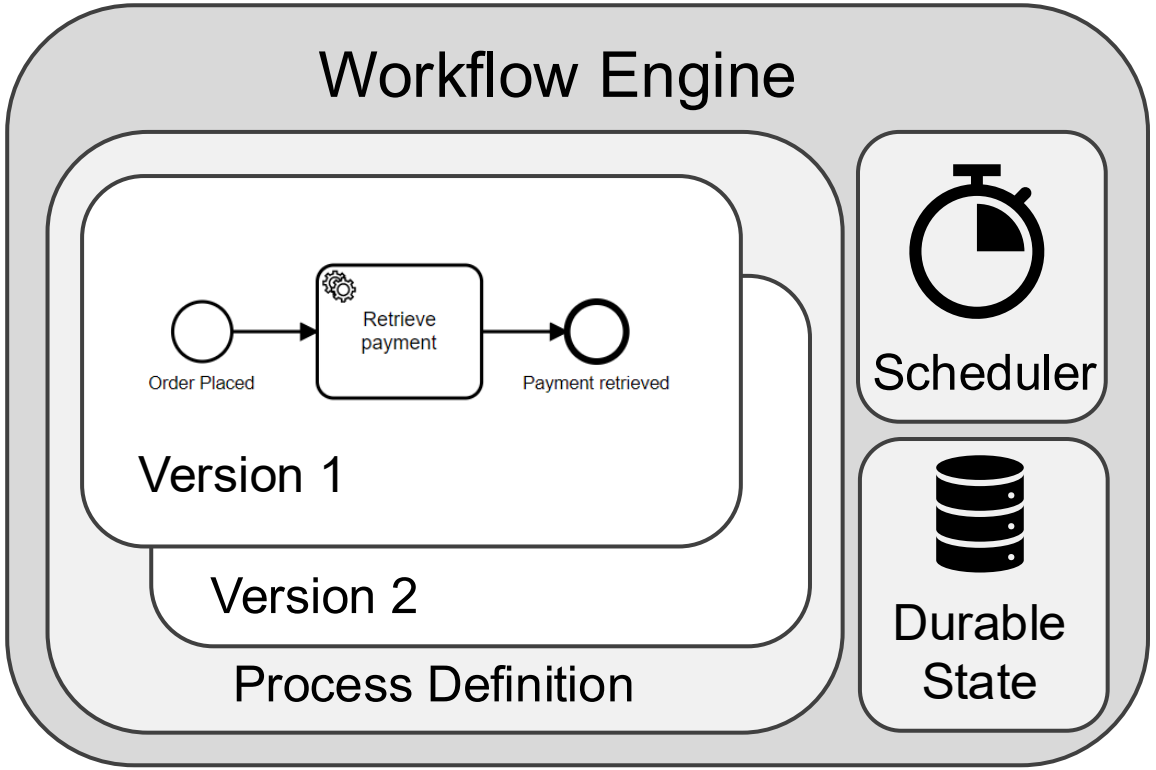
→ Operational resilience improves

Process Versioning

→ Deployments do not break running workflows

→ Infrastructure becomes more central

Operational & Governance Benefits



Process Visibility

- Graphical models improve shared understanding
- Execution state is inspectable at runtime

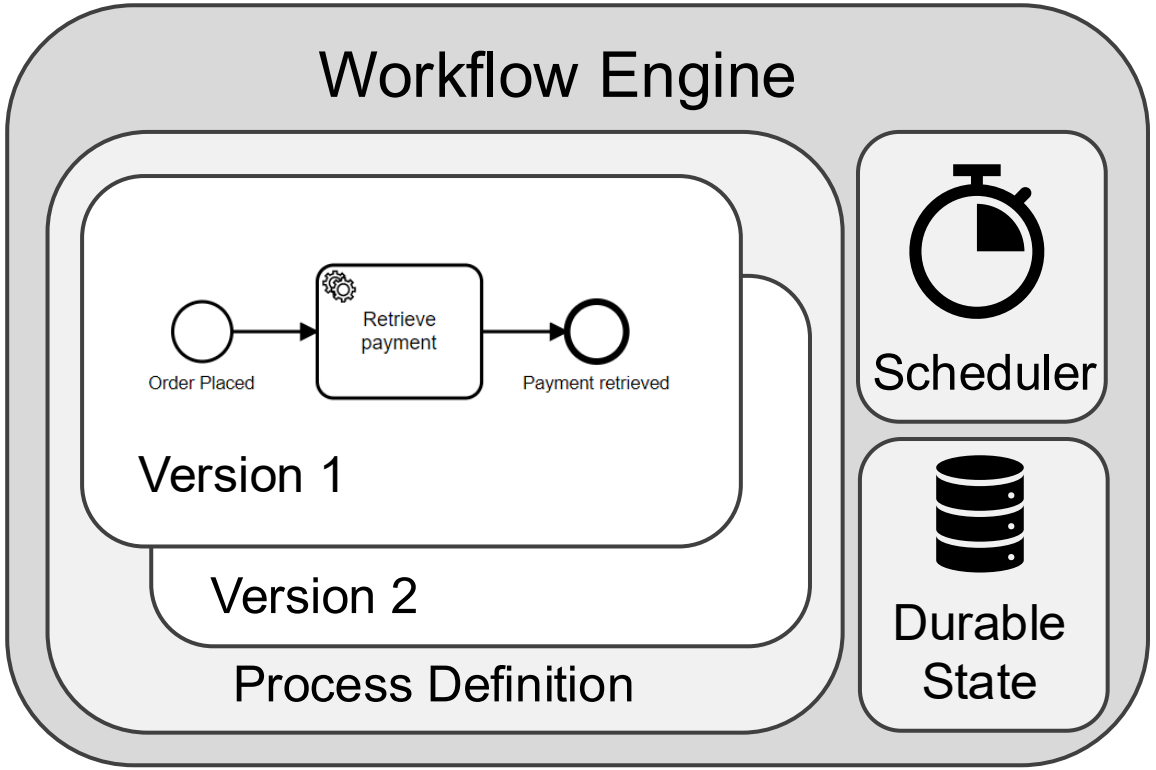
Audit & Traceability

- Full execution history per instance
- Supports compliance and debugging

Integrated Tooling

- Modeling, monitoring, and operational dashboards
- Engine becomes part of the platform

Human-in-the-Loop Workflows



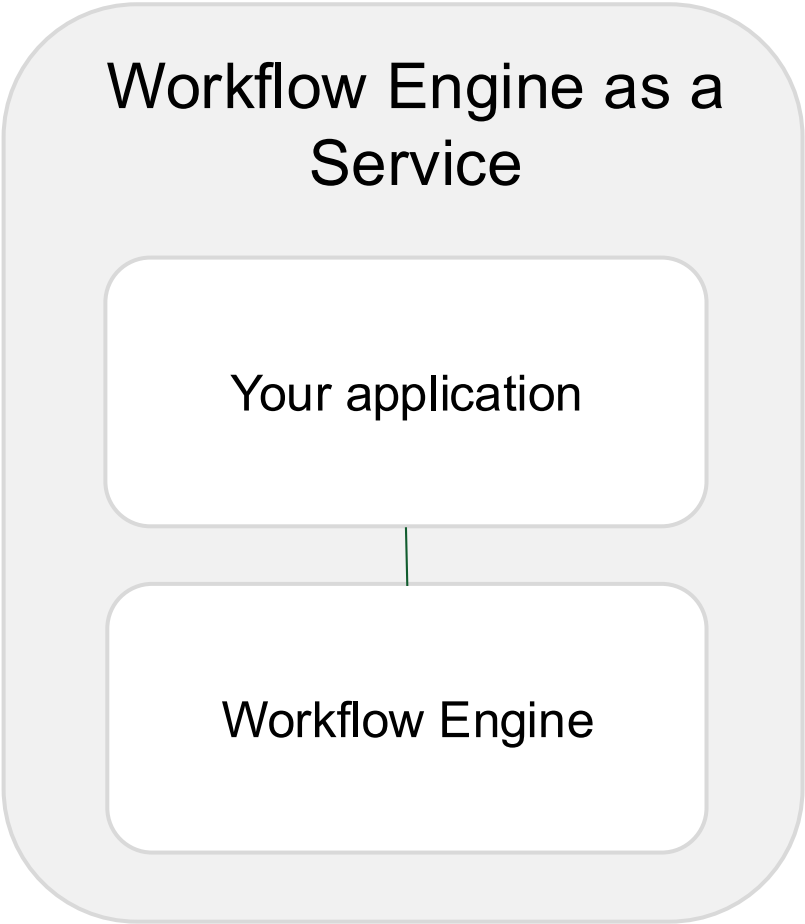
Human Tasks & Waiting States

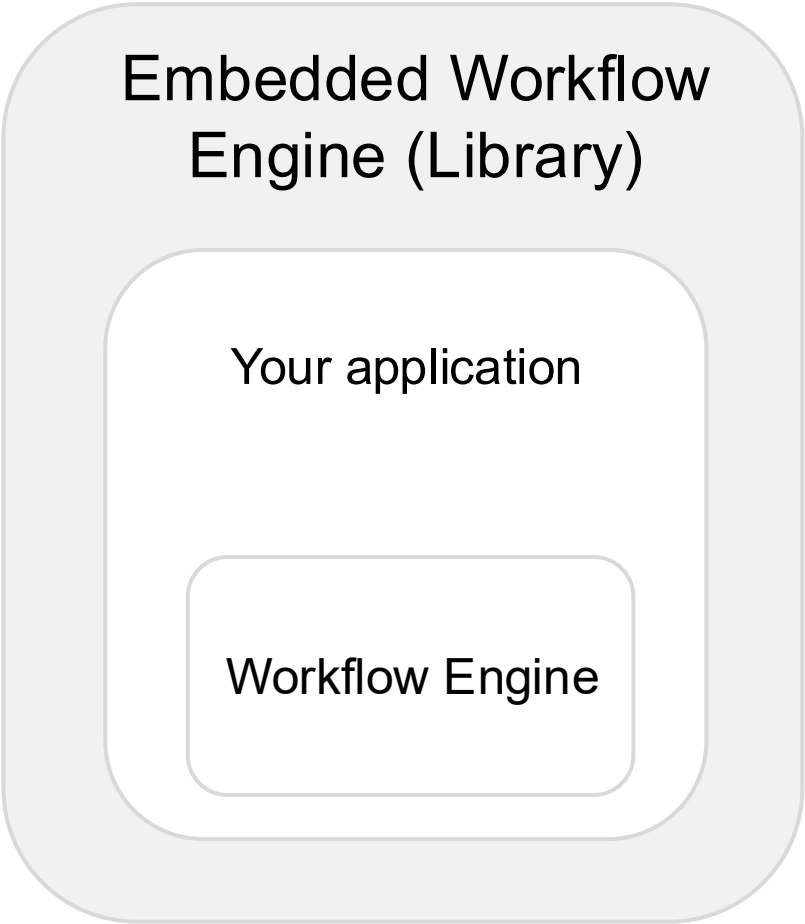
- Workflows can pause for manual decisions
- Supports approvals, reviews, escalations

Long-Running Interaction

- Processes may span hours, days, or weeks
- Engine persists state between steps

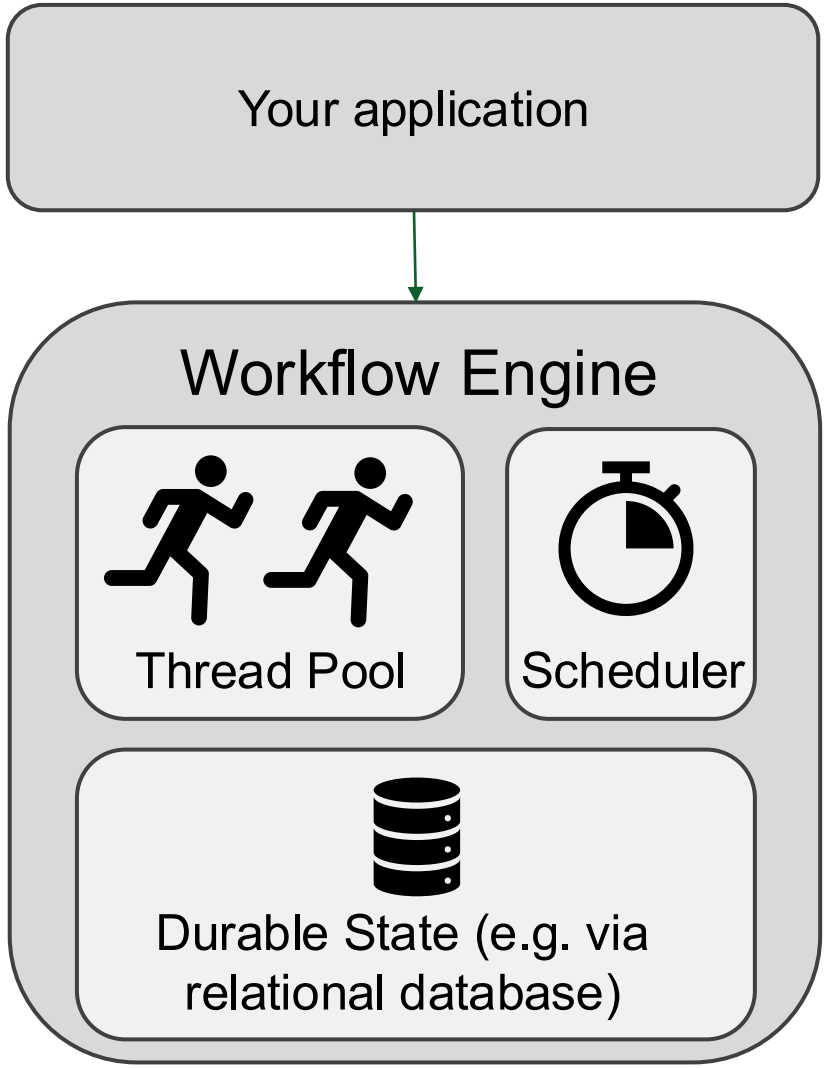
Architectural Patterns for Using an Engine





Remember the service versus library trade-off from last semester

How Engines Persist Workflow State



Process Definition Table

ID	Process definition	Version	Process model
12	Order	1.0	<bpmn ..
45	Order	2.0	<bpmn ..
82	Payment	1.0	<bpmn ..

Process Instance Table

Instance ID	Process definition	Activity	…
7421	45	Wait for payment	...
7891	45	Ship goods	...
1090	87	Customer check	...

A Workflow Model is not Enough

Process Solution

Process Definitions

BPMN / workflow model

API Layer

REST endpoint, user forms

Integration Code

Adapters, exeternal calls

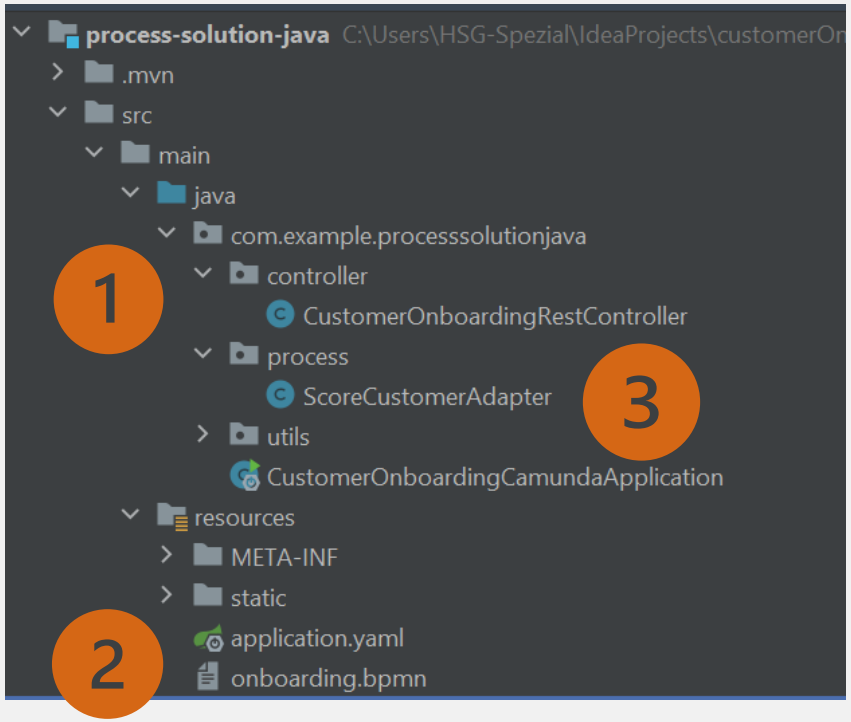
Business logic

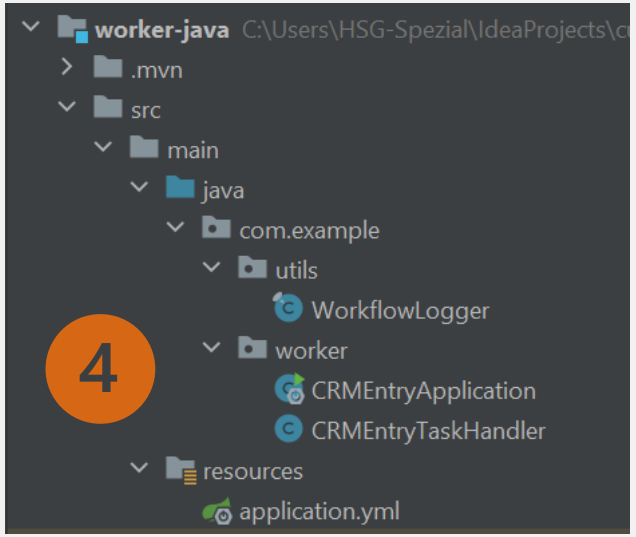
Domain logic, validations

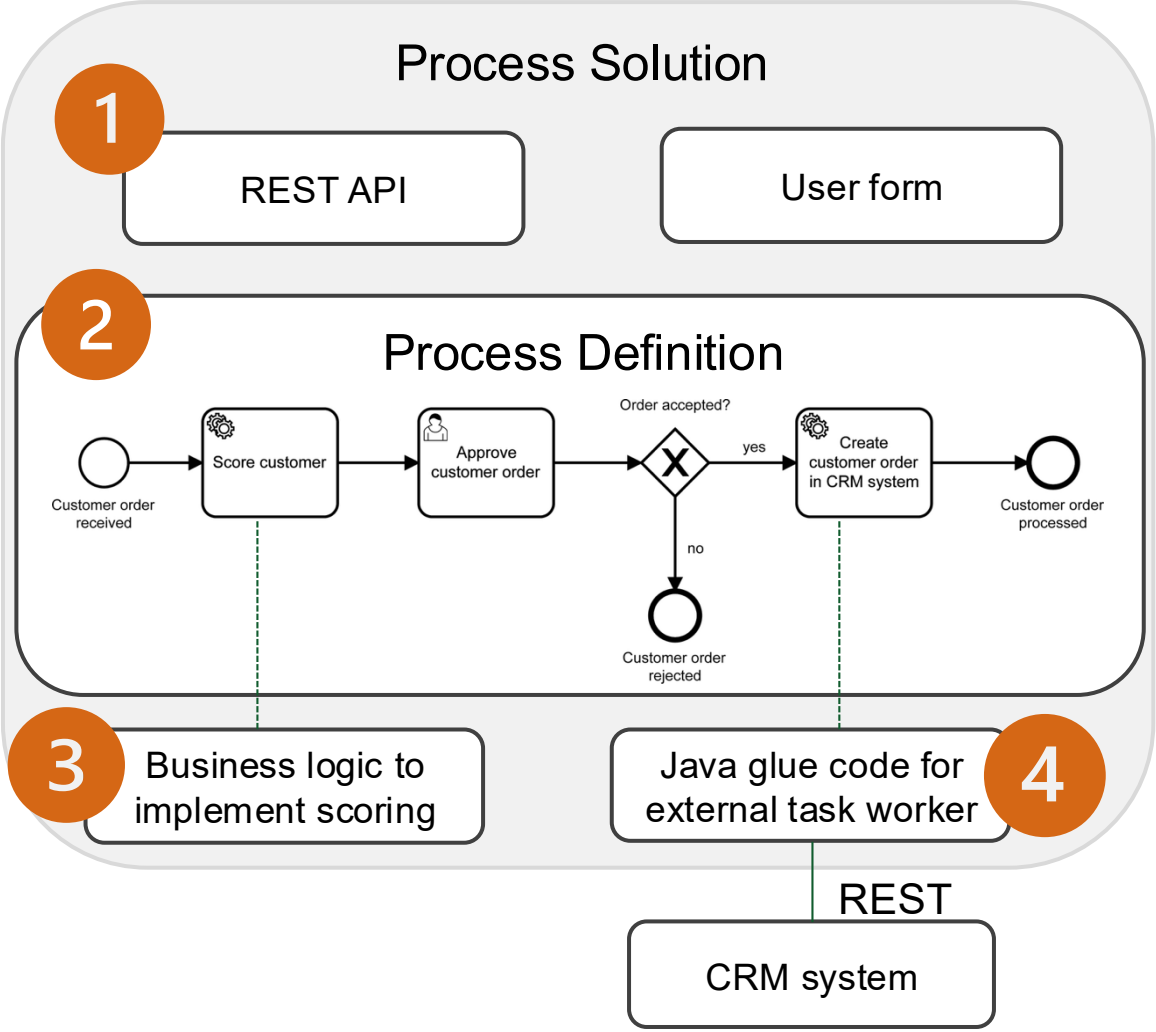
Application responsibilities

A Process Solution is a Development Project

Java Maven project (using Spring Boot)

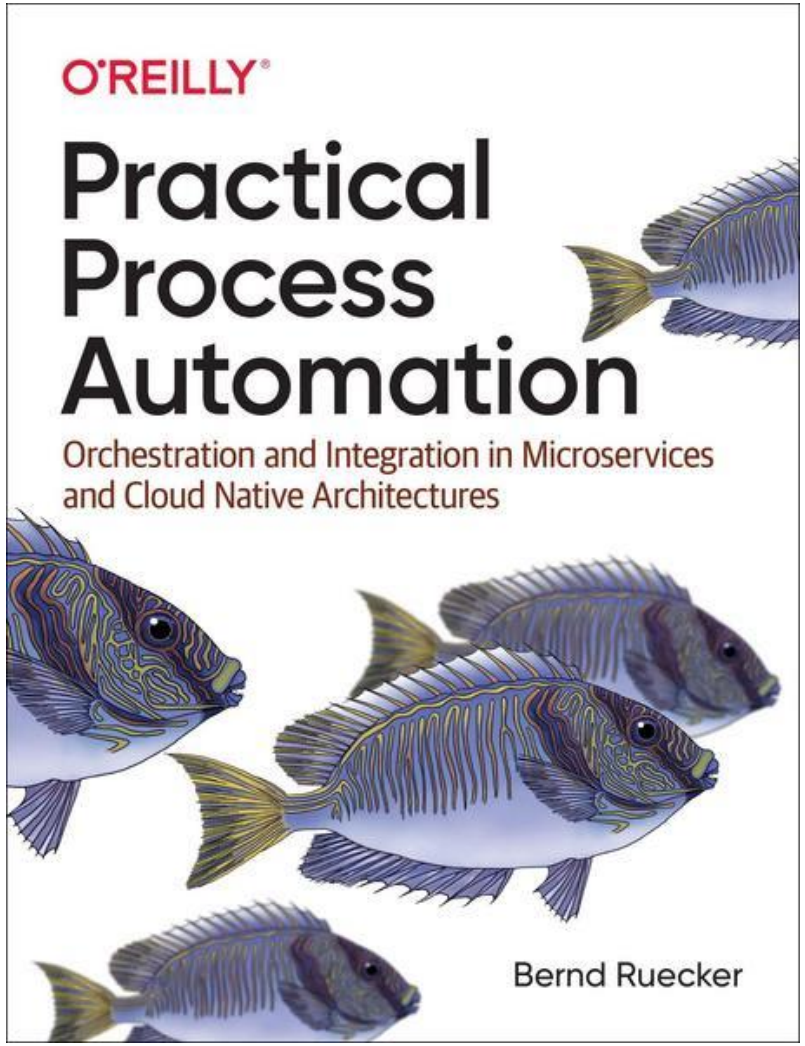






Adaptation of Process solution example for customer onboarding from Practical Process Automation.

Further References



Based on Chapter 2 – Workflow Engines and Process Solutions

Agenda

- Workflow Engines: When and Why?
- **Development of Process Solutions**
- Orchestrating Everything

BPMN for specifying executable process definitions

- Token Concept
- Core modeling elements
  - Start and end events
  - Sequence Flow
  - Tasks (service tasks, user tasks, business rule tasks, script tasks)
  - Gateways (exclusive, parallel)
  - Events (intermediate and boundary events, different event types)

BPMN – A Short Primer

- BPMN has been an ISO Standard since 2013
- BPMN can be used to define a graphical model and *execution semantics*
- Stored as XML and can be executed directly by a workflow engine, which stores persistent state of running instances

Source: BPMN and Microservice Orchestration

Universität St.Gallen

BPMN Process as XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<bpmn:definitions xmlns:bpmn="http://www.omg.org/spec/BPMN/20100524/MODEL" xmlns:bpmndi="http://www.omg.org/spec/BPMN/20100524/DI"
xmlns:dc="http://www.omg.org/spec/DD/20100524/DC" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:camunda="http://camunda.org/schema,
xmlns:di="http://www.omg.org/spec/DD/20100524/DI" xmlns:modeler="http://camunda.org/schema/modeler/1.0" id="Definitions_0bykl04"
targetNamespace="http://bpmn.io/schema/bpmn" exporter="Camunda Modeler" exporterVersion="4.11.1" modeler:executionPlatform="Camunda Platform"
modeler:executionPlatformVersion="7.15.0">
```

Execution semantics understood by workflow engine

```xml
<bpmn:process id="OrderFulfillment" isExecutable="true">
  <bpmn:startEvent id="Event_OrderPlaced" name="Order Placed">
    <bpmn:outgoing>Flow_02ka33a</bpmn:outgoing>
  </bpmn:startEvent>
  <bpmn:sequenceFlow id="Flow_02ka33a" sourceRef="Event_OrderPlaced" targetRef="Task_RetrievePayment" />
  <bpmn:endEvent id="Event_PaymentRetrieved" name="Payment retrieved">
    <bpmn:incoming>Flow_0ddj1wi</bpmn:incoming>
  </bpmn:endEvent>
  <bpmn:serviceTask id="Task_RetrievePayment" name="Retrieve payment" camunda:delegateExpression="#{RetrievePayment}">
    <bpmn:incoming>Flow_02ka33a</bpmn:incoming>
    <bpmn:outgoing>Flow_0ddj1wi</bpmn:outgoing>
  </bpmn:serviceTask>
  <bpmn:sequenceFlow id="Flow_0ddj1wi" sourceRef="Task_RetrievePayment" targetRef="Event_PaymentRetrieved" />
</bpmn:process>
```

Graphical layout information

```xml
<bpmndi:BPMNDiagram id="BPMNDiagram_1">
  <bpmndi:BPMNPlane id="BPMNPlane_1" bpmnElement="OrderFulfillment">
    <bpmndi:BPMNEdge id="Flow_0ddj1wi_di" bpmnElement="Flow_0ddj1wi">
      <di:waypoint x="330" y="110" />
      <di:waypoint x="382" y="110" />
    </bpmndi:BPMNEdge>
    <bpmndi:BPMNEdge id="Flow_02ka33a_di" bpmnElement="Flow_02ka33a">
      <di:waypoint x="178" y="110" />
      <di:waypoint x="230" y="110" />
    </bpmndi:BPMNEdge>
    <bpmndi:BPMNShape id="Event_0mmbodz_di" bpmnElement="Event_OrderPlaced">
      <dc:Bounds x="142" y="92" width="36" height="36" />
      <bpmndi:BPMNLabel>
        <dc:Bounds x="127" y="135" width="65" height="14" />
```

Core Concepts

Sequence Flow
represents the order in which steps in the process happen

End Event
The process instance ends here

Start Event
The flow begins here when a new process instance is stared

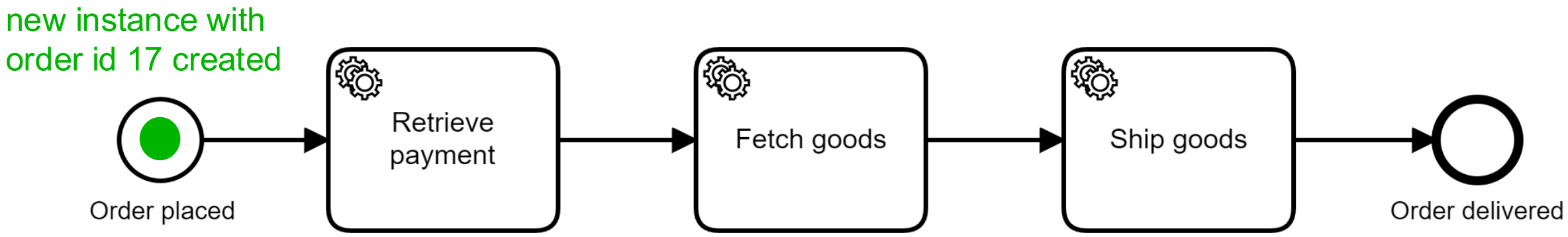
Tasks
Are atomic units of work

The Token Concept

- According to the BPMN spec the token is "*a theoretical concept that is used as an aid to define the behavior of a process that is being performed*"
- A process instance can be imagined as a token moving through a process model
- Every order will have its own process instance and thus its own token
- The token indicates where an instance currently is in the process, and it moves through the process upon completion of a task
- The status of these tokens is typically persisted by the workflow engine

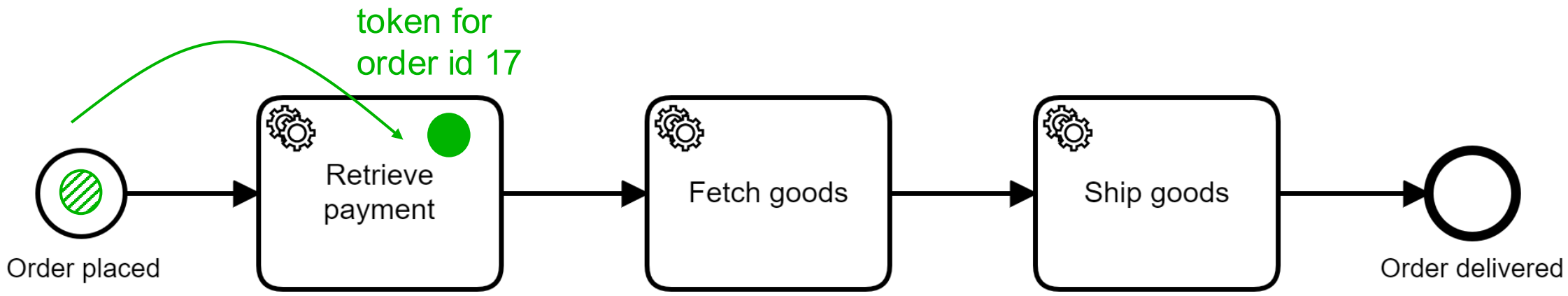
The Token Concept

- When starting an instance, a token is spawned at the start event



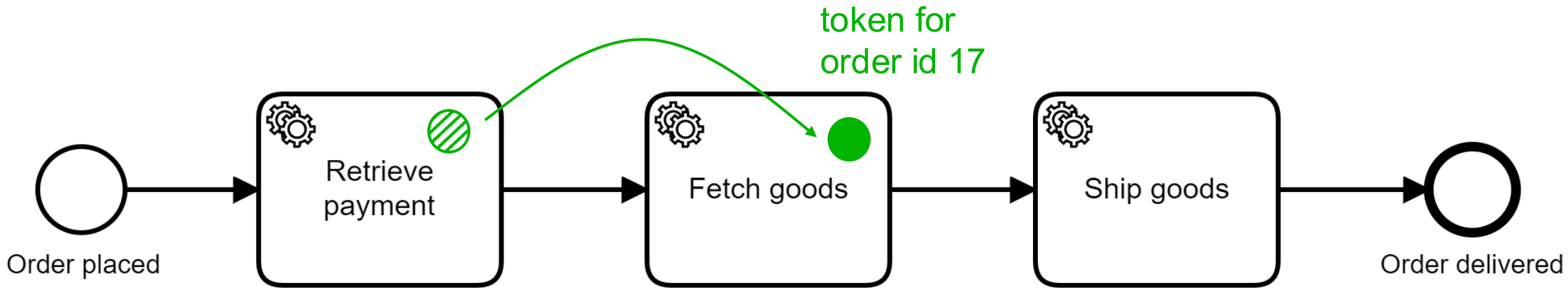
The Token Concept

- When starting an instance, a token is spawned at the start event
- The token moves forward in the process upon completion of a task



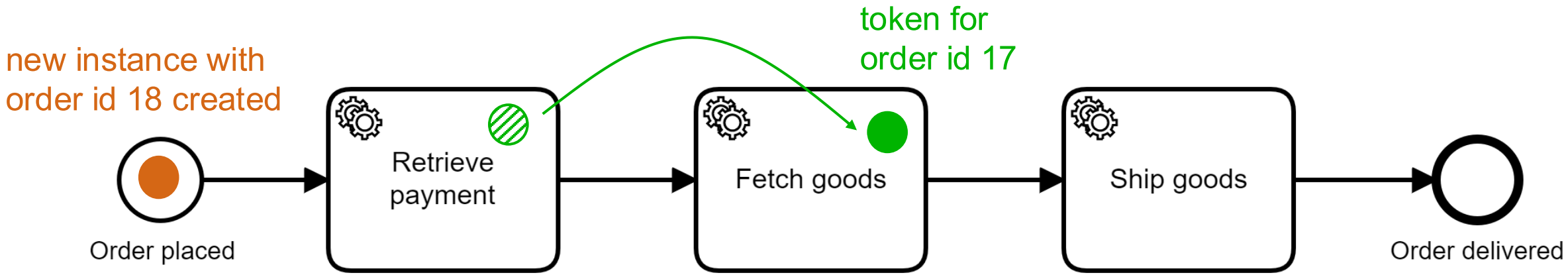
The Token Concept

- When starting an instance, a token is spawned at the start event
- The token moves forward in the process upon completion of a task



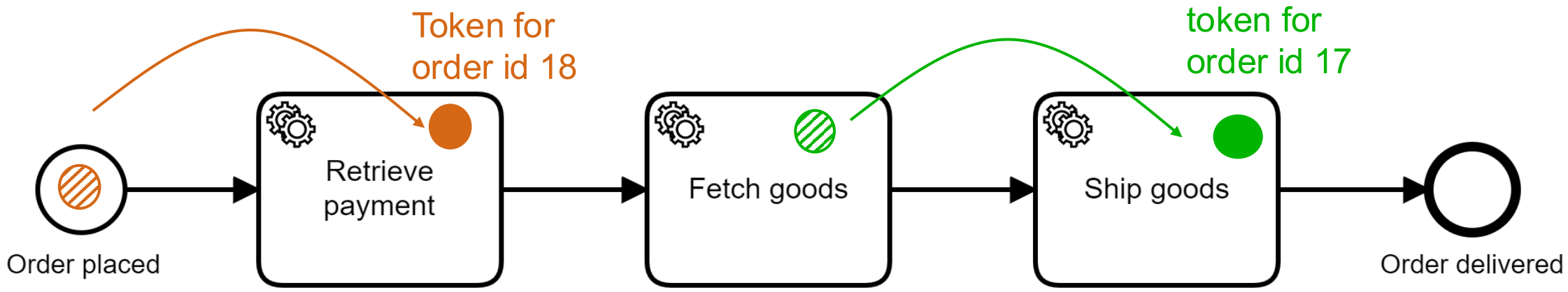
The Token Concept

- When starting an instance, a token is spawned at the start event
- The token moves forward in the process upon completion of a task
- For every instance that gets started, a token is created resulting into multiple tokens flowing through the model at the same time



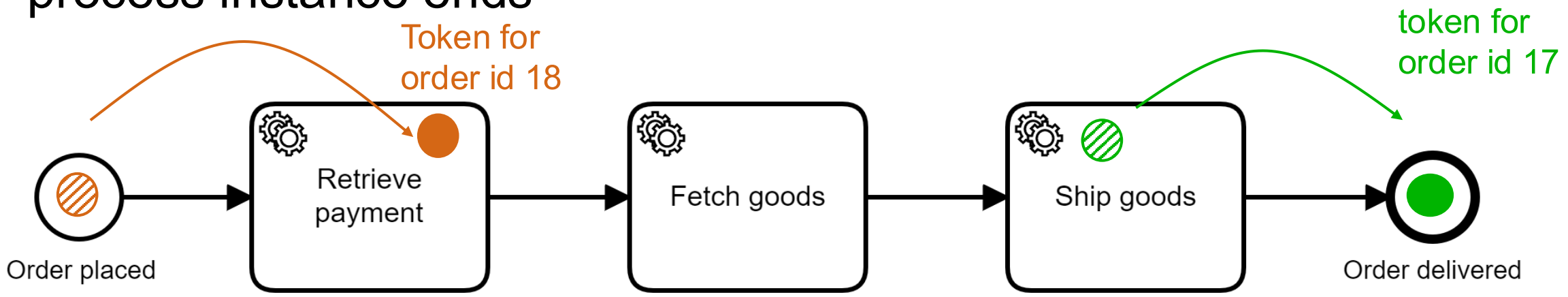
The Token Concept

- When starting an instance, a token is spawned at the start event
- The token moves forward in the process upon completion of a task
- For every instance that gets started, a token is created resulting into multiple tokens flowing through the model at the same time



The Token Concept

- When starting an instance, a token is spawned at the start event
- The token moves forward in the process upon completion of a task
- For every instance that gets started, a token is created resulting into multiple tokens flowing through the model at the same time
- When reaching the end event, the token is consumed, and the process instance ends



Tasks

- Tasks are atomic units of work
- When a token arrives at a task it stops until the task is completed; only then it continues on the outgoing sequence flow

Task Type	Description
Service Task	Executes some software functionality such as a calling a service, a microservice, a method, a function, etc.
User Task	Create to-do items for humans in a tasklist; the workflow engine waits until a human worker has ticked off the to-do item
Decision Task	Can involve a decision engine to evaluate a rule table
Script Task	Executes a script in a defined scripting language

Universität St.Gallen

Tasks

```java
@Service("RetrievePayment")
public class RetrievePayment implements JavaDelegate {
    private final Logger logger = LoggerFactory.getLogger(this.getClass());

    @Override
    public void execute(DelegateExecution execution) {
        WorkflowLogger.info(logger,  method: "retrievePayment",  message: "Paymen
    }
}
```

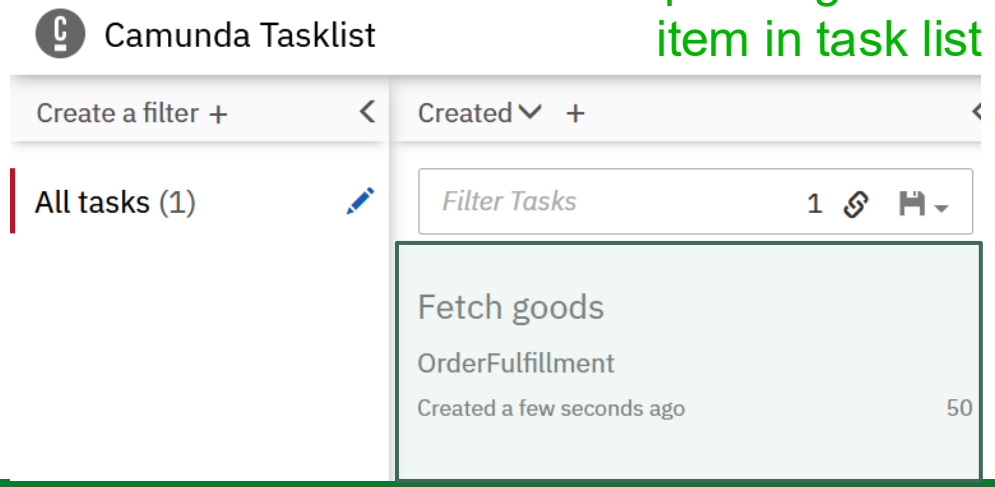
Corresponding glue code

Service task
indicated by the gear wheel icon

token for order id 129

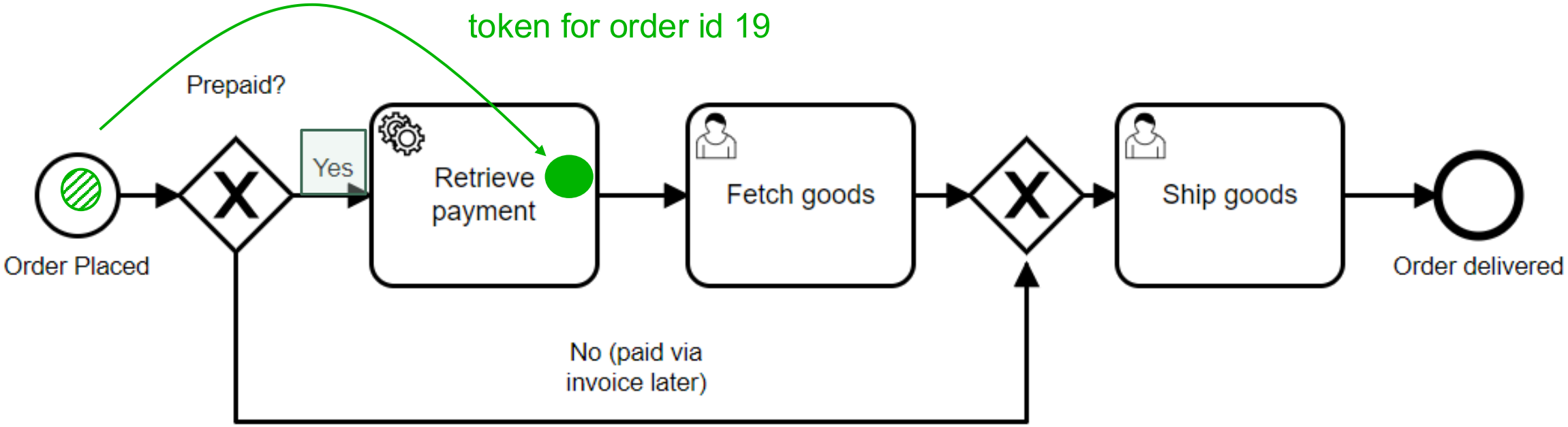
User task
indicated by the human icon

Corresponding to-do item in task list



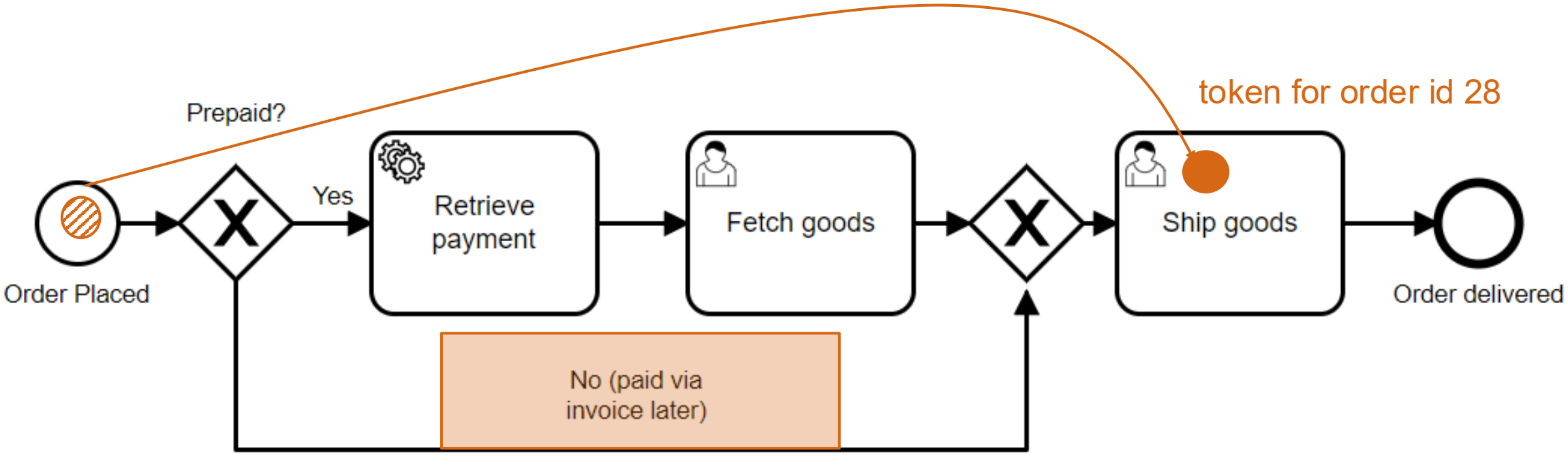
Gateways for Controlling the Flow
Example of an Exclusive Gateway

- An exclusive gateway chooses exactly one sequence flow out of many depending on data



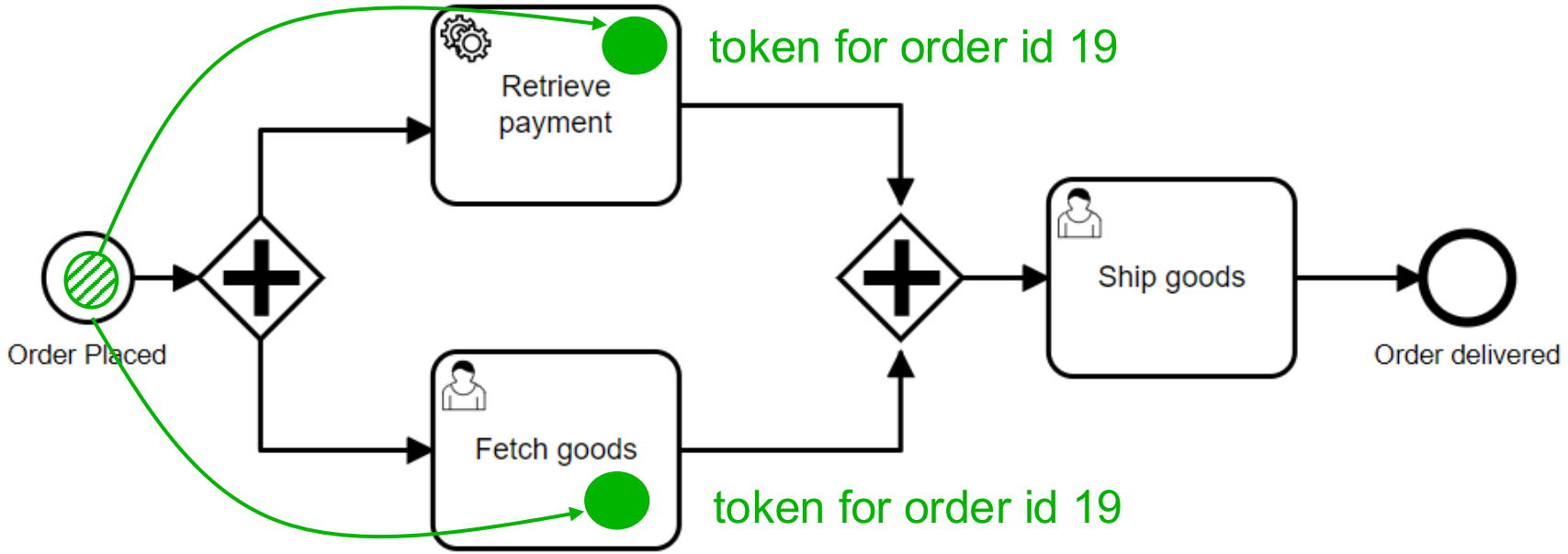
Gateways for Controlling the Flow
Example of an Exclusive Gateway

- An exclusive gateway chooses exactly one sequence flow out of many depending on data



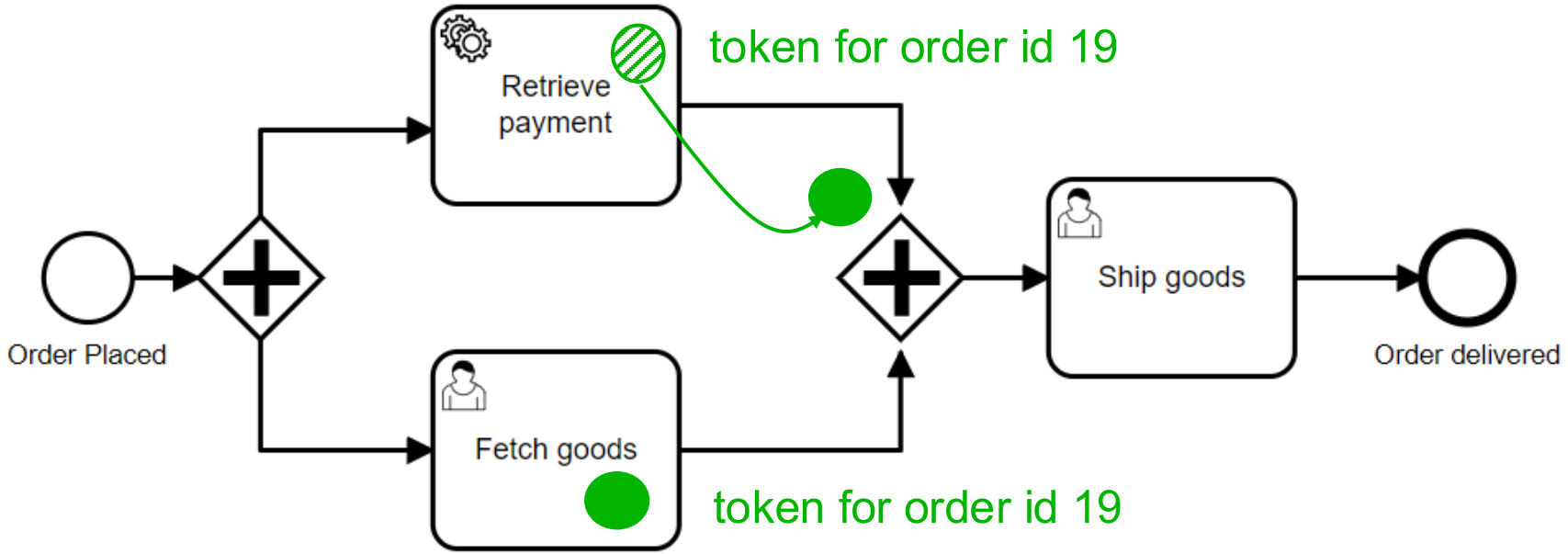
Gateways for Controlling the Flow
Example of a Parallel Gateway

- A parallel gateway produces new tokens by activating multiple sequence flows in parallel (which does not necessarily mean that tasks run concurrently). To continue with the execution of the flow after the parallel branch, all branches must be completed.



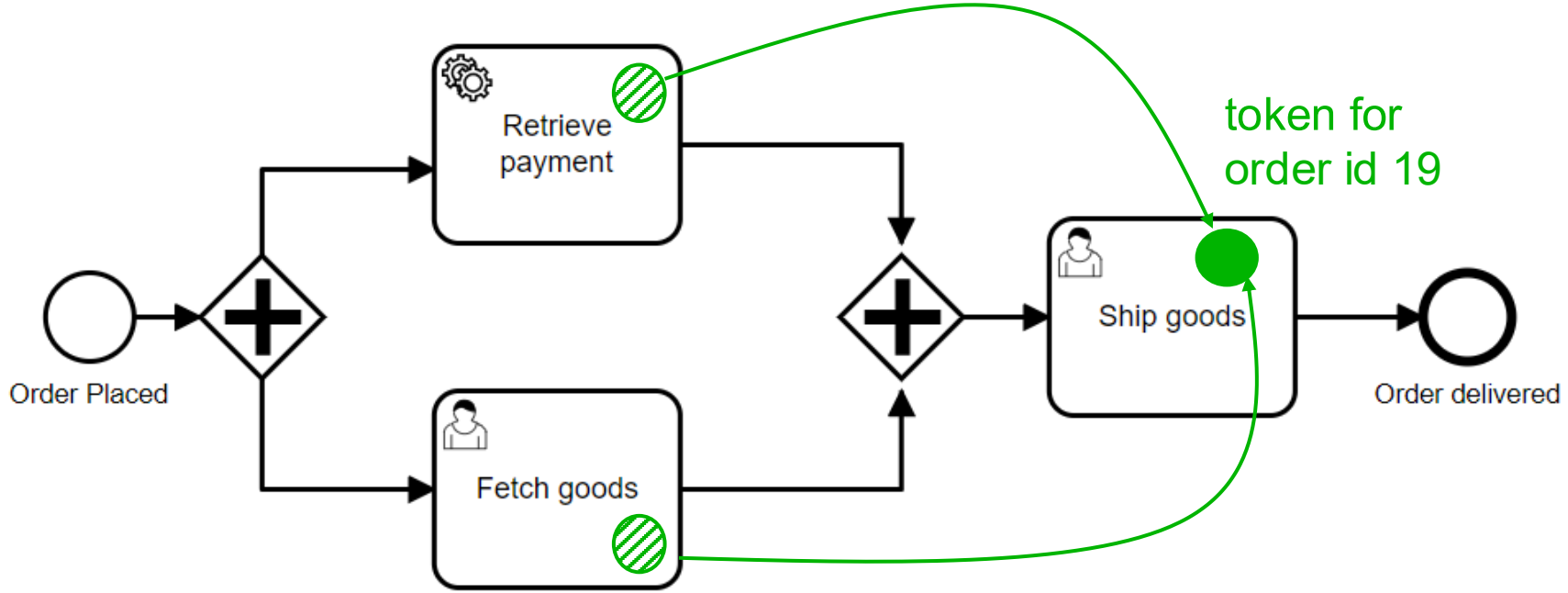
Gateways for Controlling the Flow
Example of a Parallel Gateway

- A parallel gateway produces new tokens by activating multiple sequence flows in parallel (which does not necessarily mean that tasks run concurrently). To continue with the execution of the flow after the parallel branch, all branches must be completed.



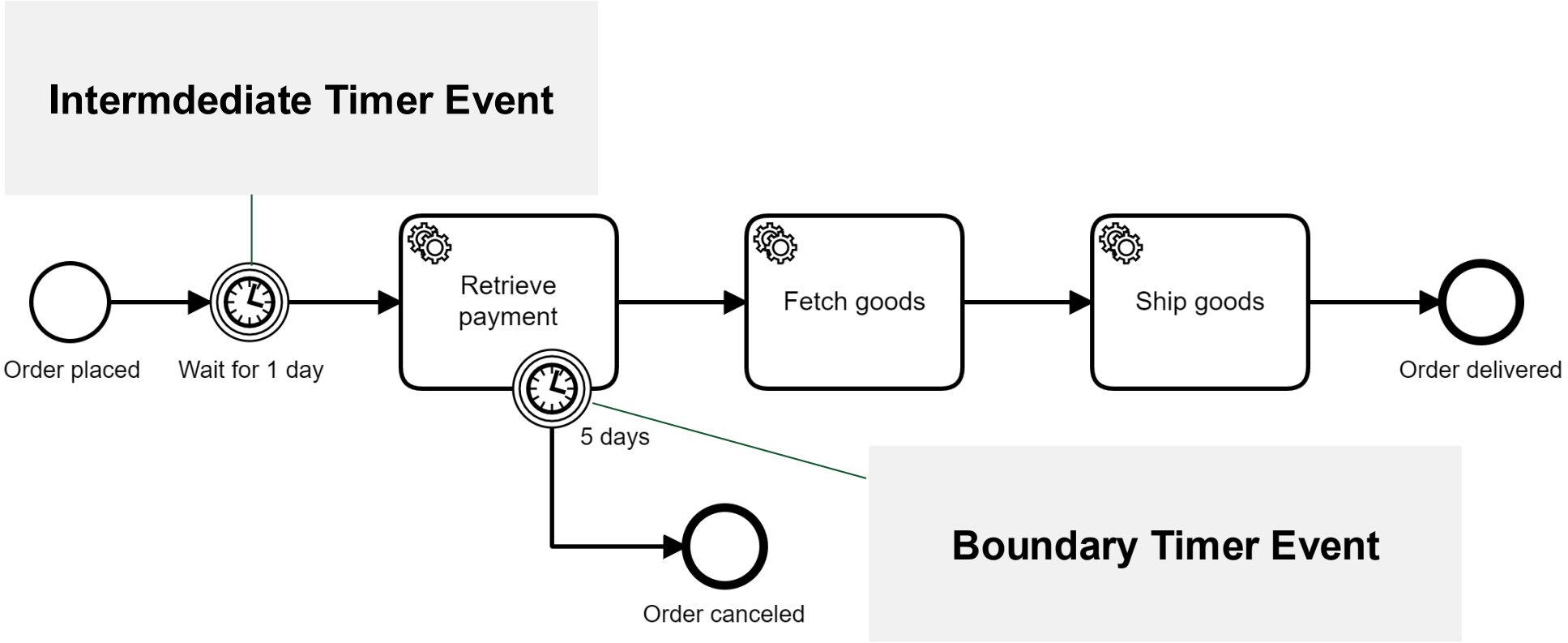
Gateways for Controlling the Flow
Example of a Parallel Gateway

- A parallel gateway produces new tokens by activating multiple sequence flows in parallel (which does not necessarily mean that tasks run concurrently). To continue with the execution of the flow after the parallel branch, all branches must be completed.



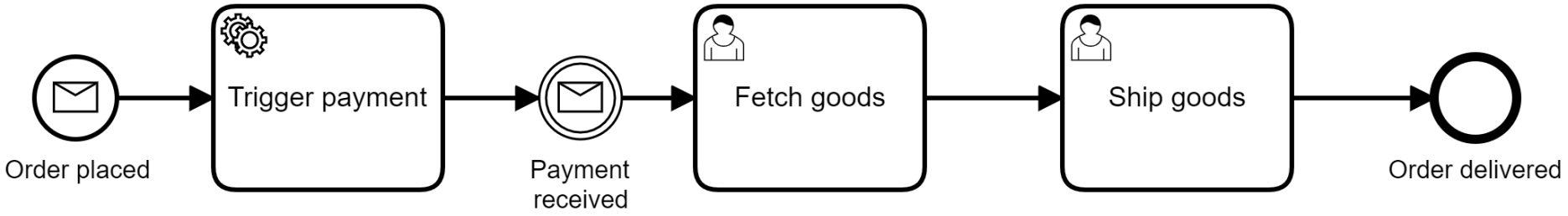
Events

- Events represent things that happen in a process
- A process can react to them by catching them



Message Events

- Message events are sent to a process instance from outside of the workflow engine
- They can start a new instance or trigger the continuation of an existing instance



- Technically message events can be a method call in your application via some REST API or a message or event in your message or event broker

Message Events

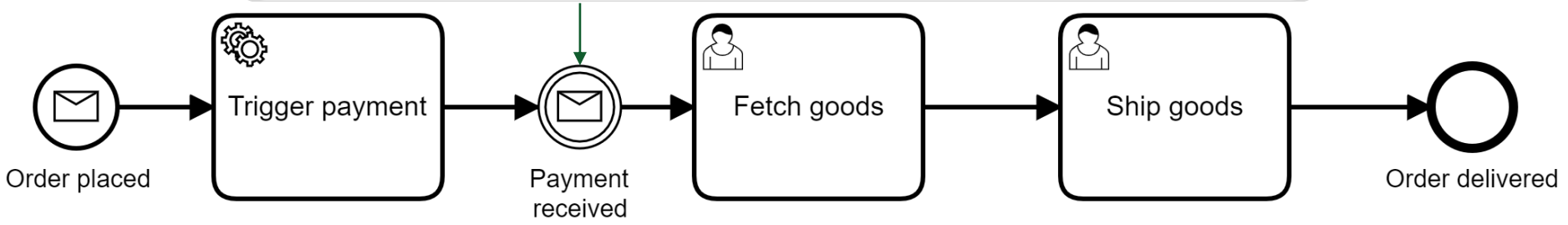
- Message events are sent to a process instance from outside of the workflow engine
- They can start a new instance or trigger the continuation of an existing instance

Message broker

Your code to receive a message

```
@KafkaListener(topics = "payment-received-topic")
public void listen(CamundaMessageDto camundaMessageDto){
    messageService.correlateMessage(camundaMessageDto, MESSAGE_INTERMEDIATE);
}
```

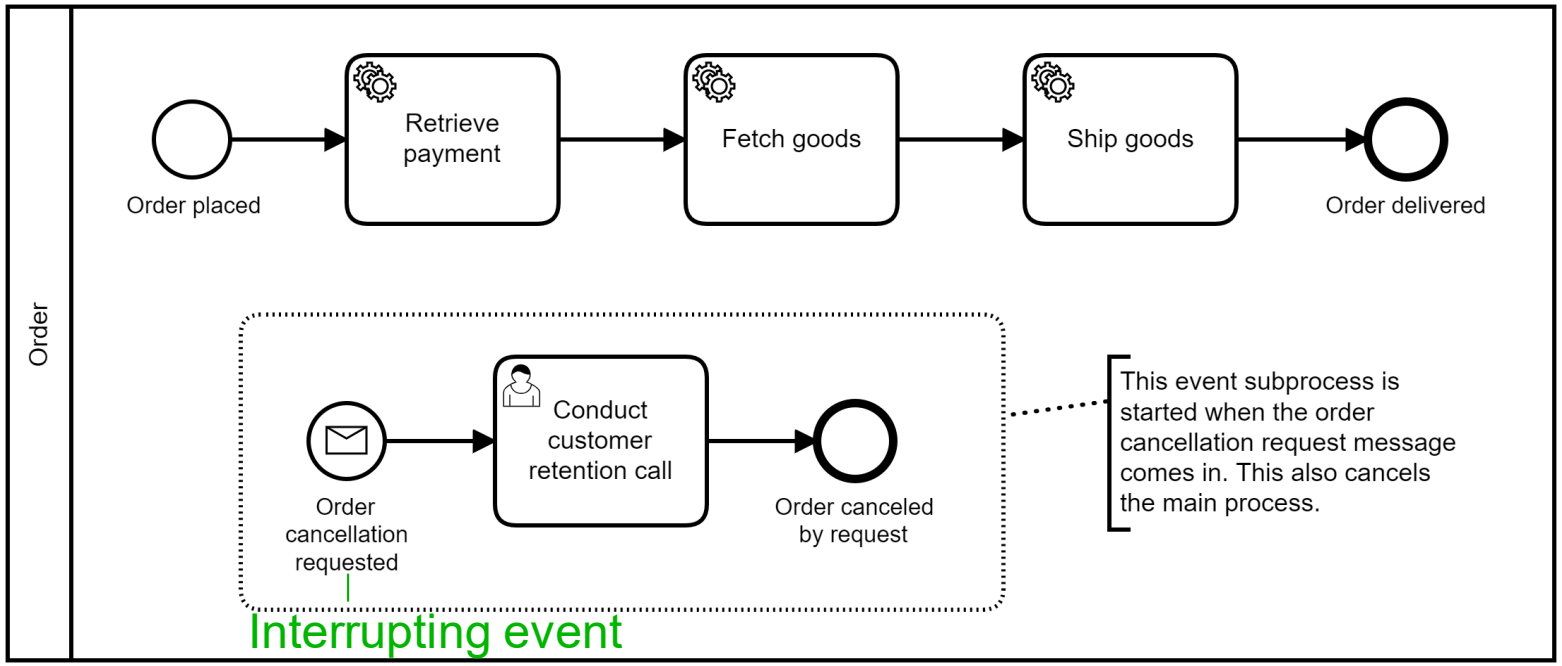
- Receive message
- Transform message
- Route message to correct waiting process instance



Example: Process reacting to messages in message broker

Message Events and Event Sub Processes

- Message Events are often used together with the event subprocess
- Event subprocesses can be used to interrupt a process instance upon receiving a message independently of which task is currently executed



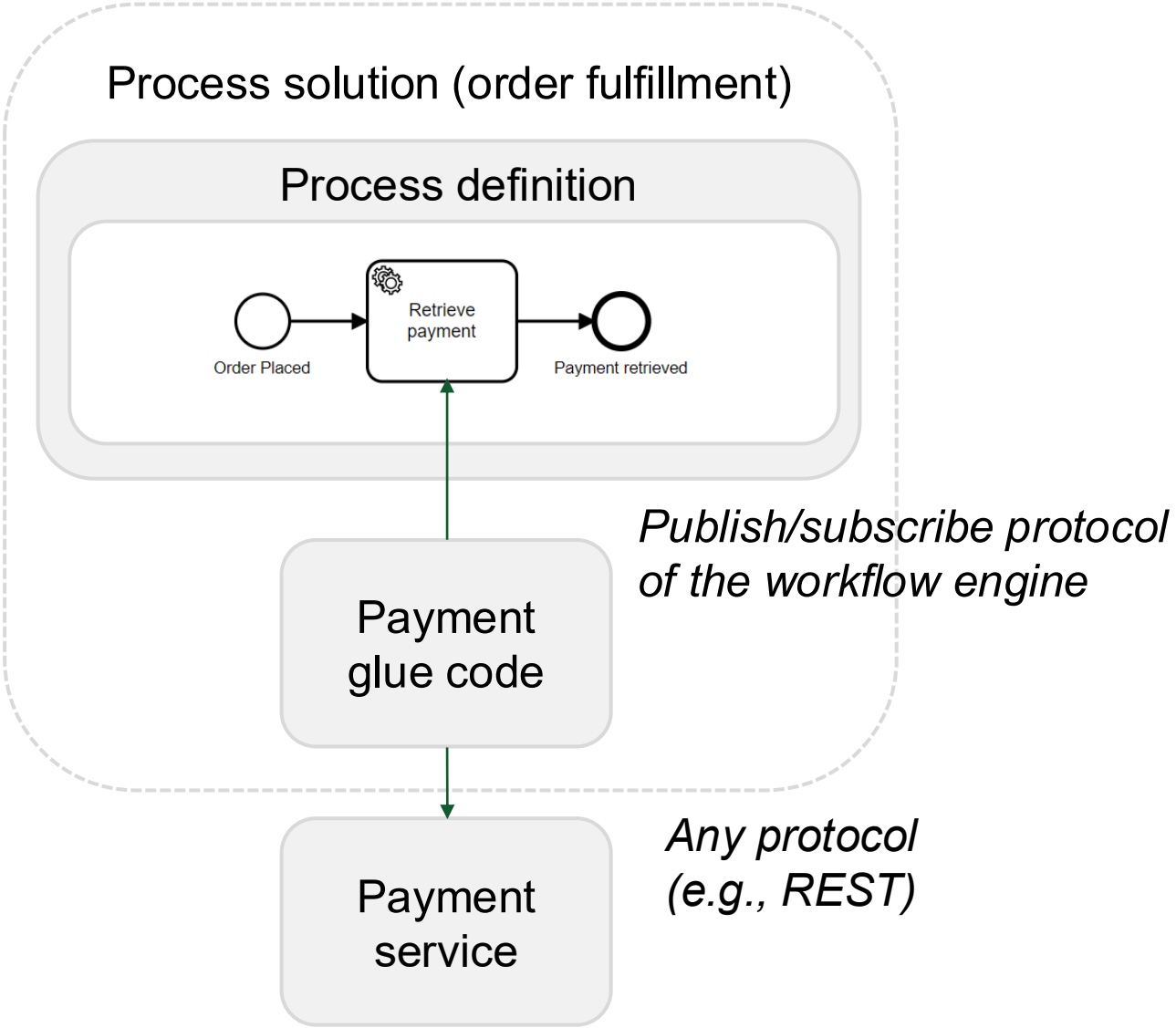
Outlook
Lecture 6 will cover how to restore consistency in such scenarios

If a cancellation request comes in it interrupts the normal flow, no matter which task is executed

Combining Process Model and Programming Code

- Publish / subscribe to a process
- Referencing code in a process model
- Using prebuilt connectors

Combining Process Model and Programming Code Publish / Subscribe to a Process



The workflow engine acts as broker and provides queues (no need for an actual broker).

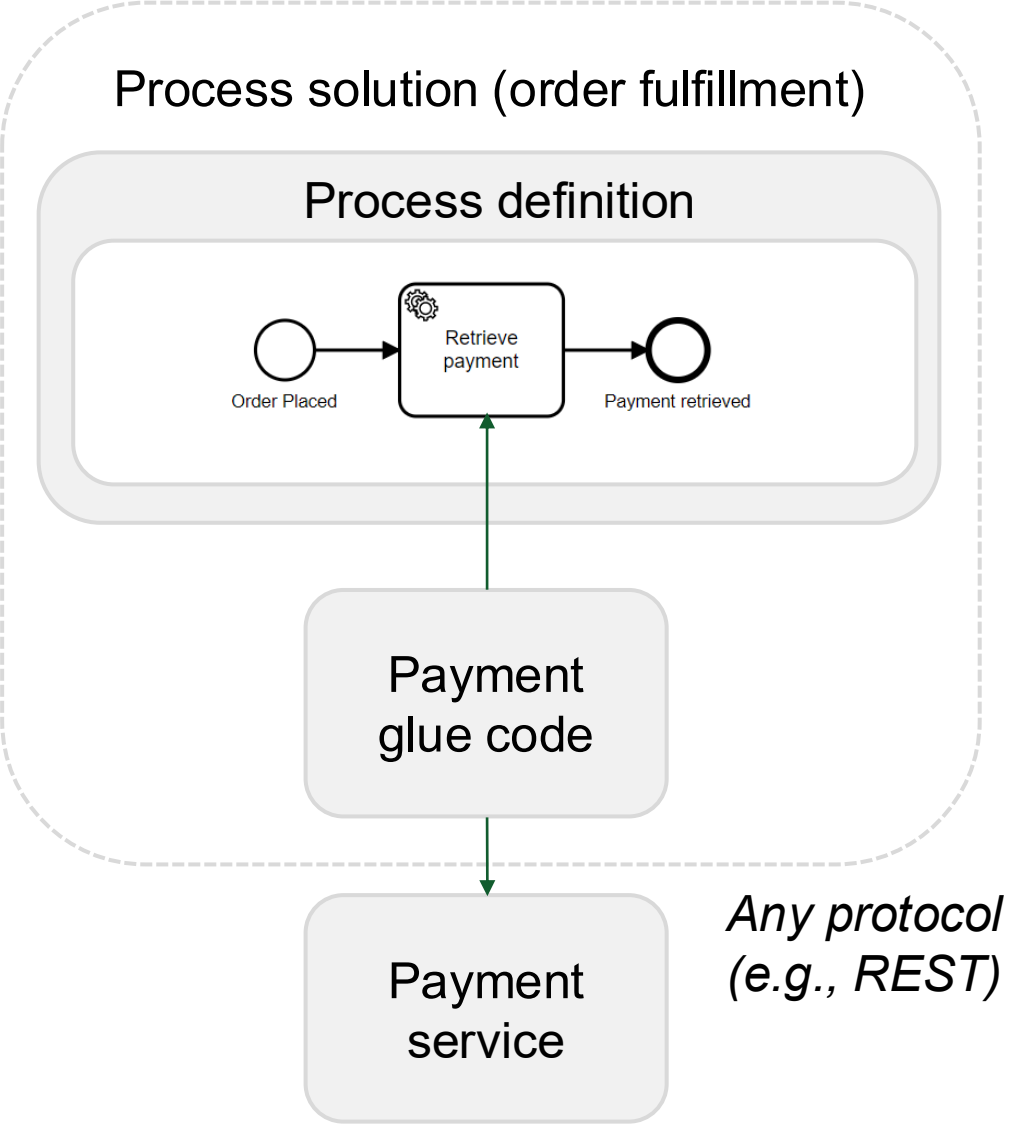
When a process instance arrives at the service task, a message is sent to the queue.

The glue code subscribes to the workflow engine and executes code whenever a new instances arrives there (often with long polling underneath).

Examples:

- Job Worker in Camunda Cloud
- External Task Worker in Camunda Platform

Combining Process Model and Programming Code
Referencing Code in a Process Model



Code can also be referenced in the process model.

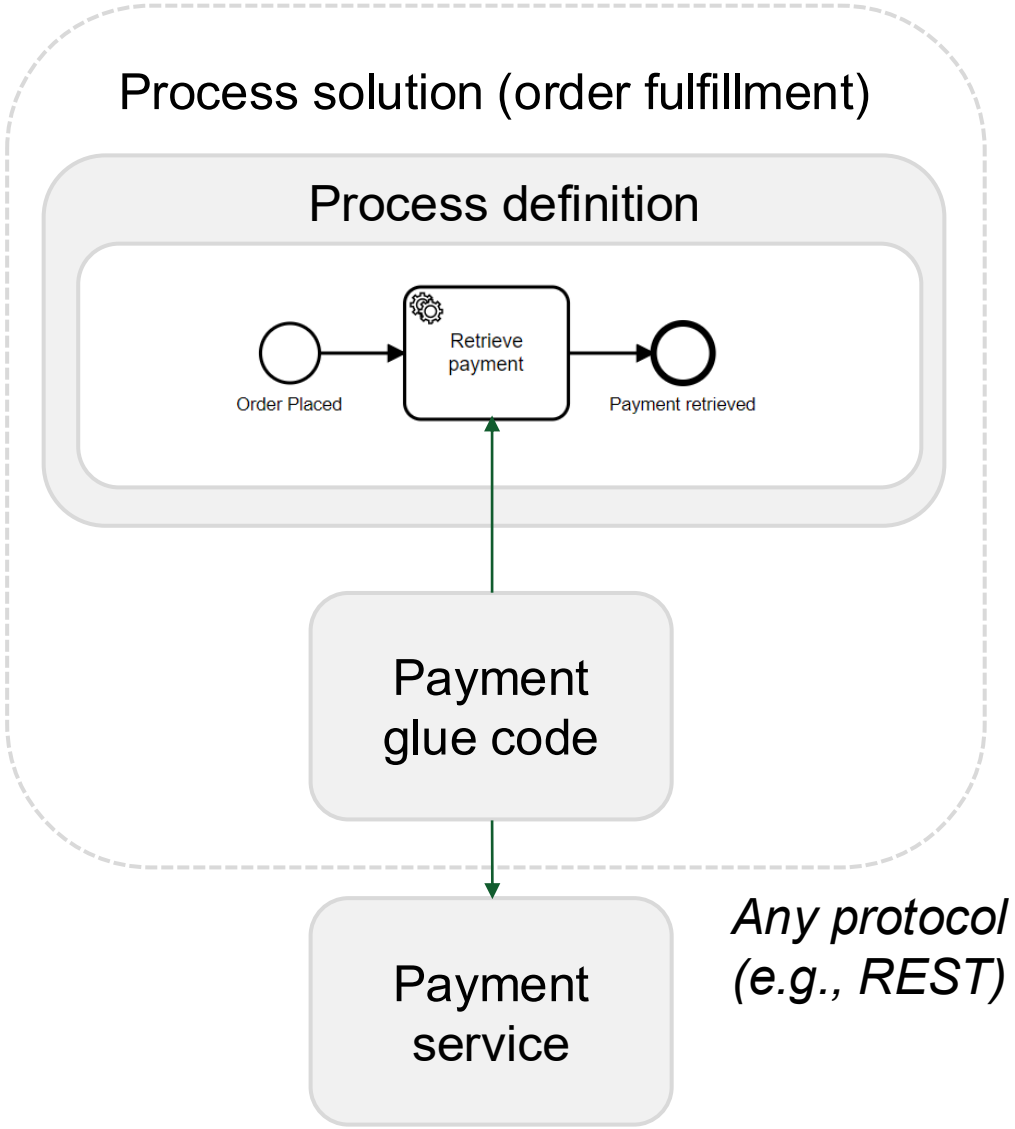
When a process instance arrives at the service task, the workflow engine executes this code.

The code is executed within the context of the workflow engine (i.e., within the thread of the engine and probably also in the same technical transaction).

Examples:

- Java Delegates in Camunda Platform

Combining Process Model and Programming Code
Referencing Code in a Process Model



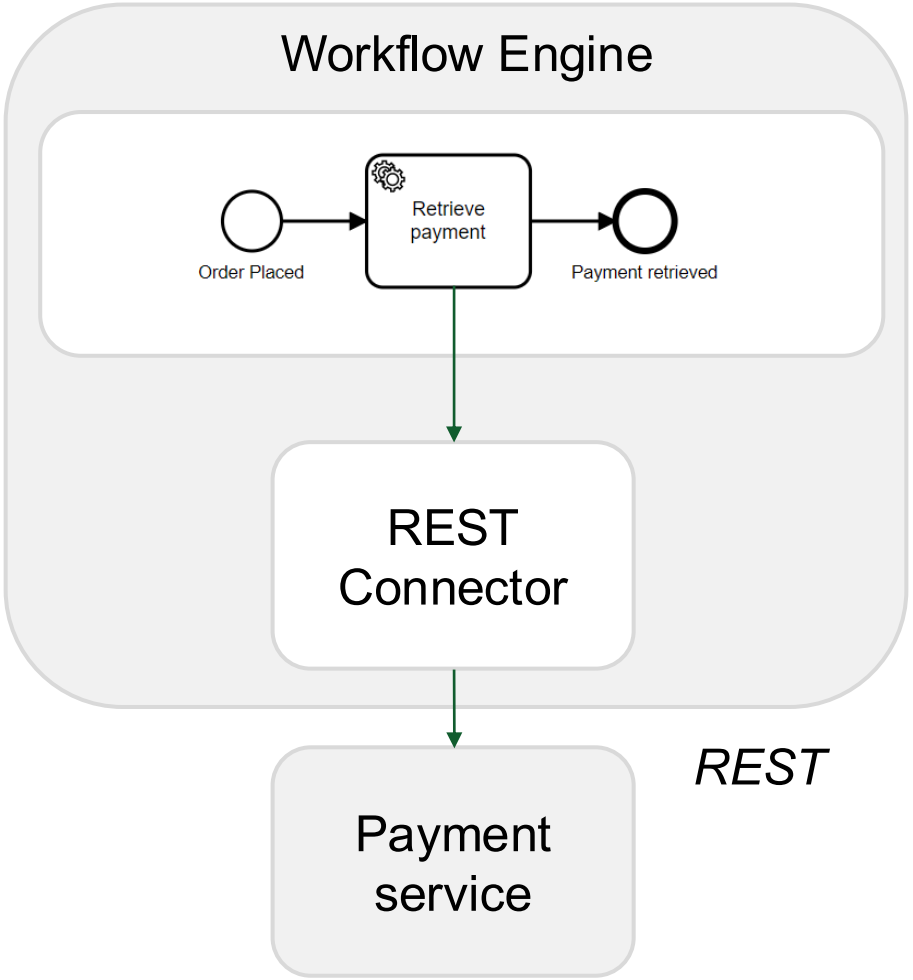

```
<bpmn:serviceTask id="Task_RetrievePayment" name="Retrieve payment"
        camunda:delegateExpression="#{RetrievePayment}">
```

```
@Service("RetrievePayment")
public class RetrievePayment implements JavaDelegate {
    private final Logger logger = LoggerFactory.getLogger(this.getClass());

    @Override
    public void execute(DelegateExecution execution) {
        WorkflowLogger.info(logger, method: "retrievePayment", message: "Paymen
    }
}
```

Example of Java Delegate in Camunda Platform

Combining Process Model and Programming Code Using Prebuilt Connectors



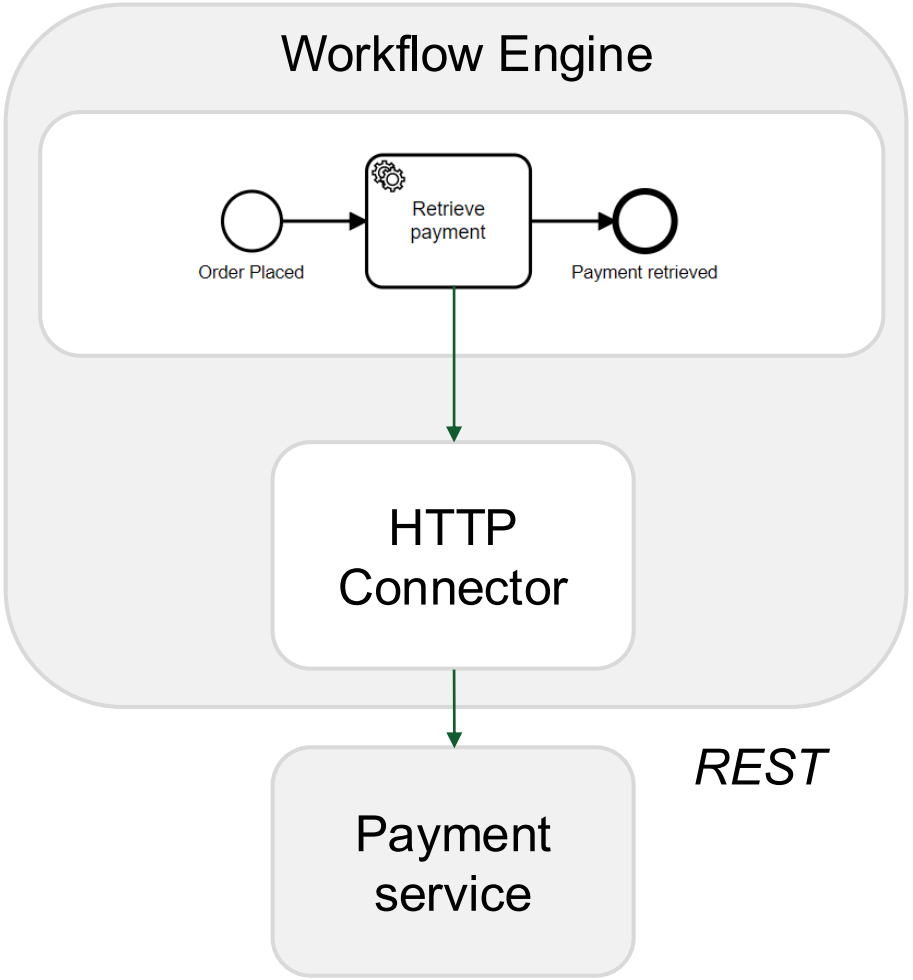
Prebuilt connectors coming with the process automation platform can be referenced and configured via the process model.

Type and power of connectors highly varies between vendors.

Examples:

- HTTP Connector in Camunda Platform
- Camunda RPA Bridge
- Camunda Cloud – AWS Lambda Worker Code

Combining Process Model and Programming Code Using Prebuilt Connectors




```
18        <bpmn:extensionElements>
19          <camunda:connector>
20            <camunda:inputOutput>
21              <camunda:inputParameter name="url">https://api.meaningcloud.com/sentim
22              <camunda:inputParameter name="method">POST</camunda:inputParameter>
23              <camunda:inputParameter name="headers">
24                <camunda:map>
25                  <camunda:entry key="Content-Type">application/json</camunda:entry>
26                </camunda:map>
27              </camunda:inputParameter>
28              <camunda:inputParameter name="payload">{ "txt":"${sentimentText}", "ke
</camunda:inputParameter>
29              <camunda:outputParameter name="response">${response}</camunda:outputPa
30              <camunda:outputParameter name="statusCode">${statusCode}</camunda:outp
31            </camunda:inputOutput>
32            <camunda:connectorId>http-connector</camunda:connectorId>
33          </camunda:connector>
34        </bpmn:extensionElements>
```

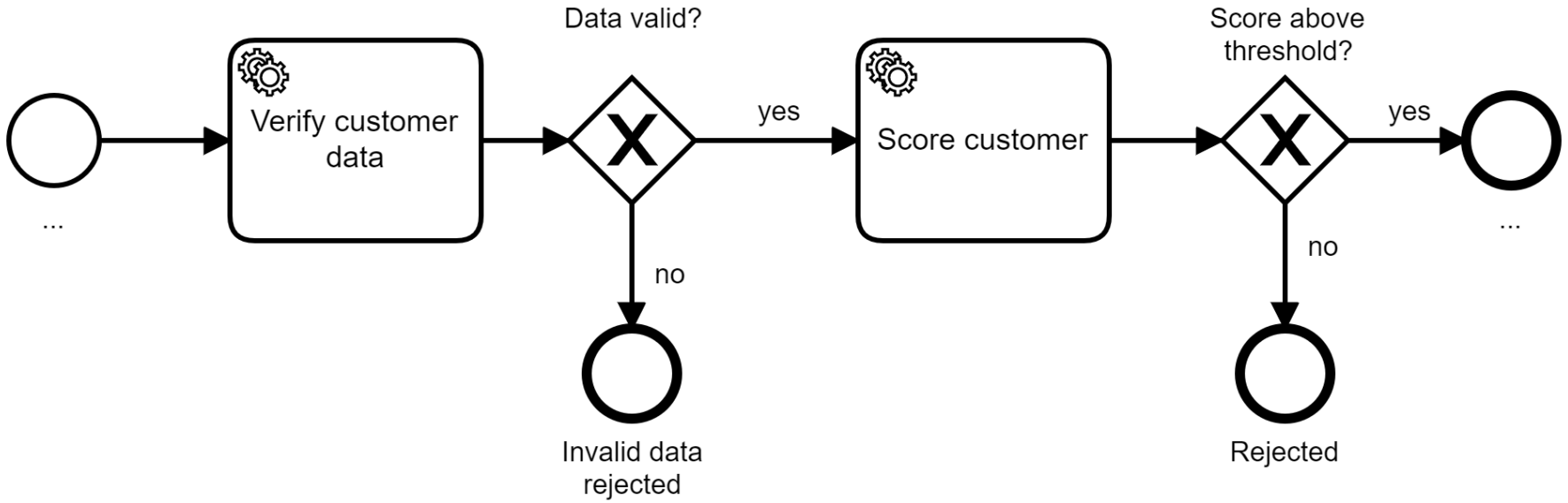
Example: HTTP Connector in Camunda Platform

Model or Code?

- Business logic belongs in code by default
- However, certain logic can be useful to have in the process model

- Questions that can guide this decision
  - Where do you need to (potentially) wait?
  - What do you discuss with regularly with other stakeholders?
  - What crosses boundaries?

Model or Code? An Example

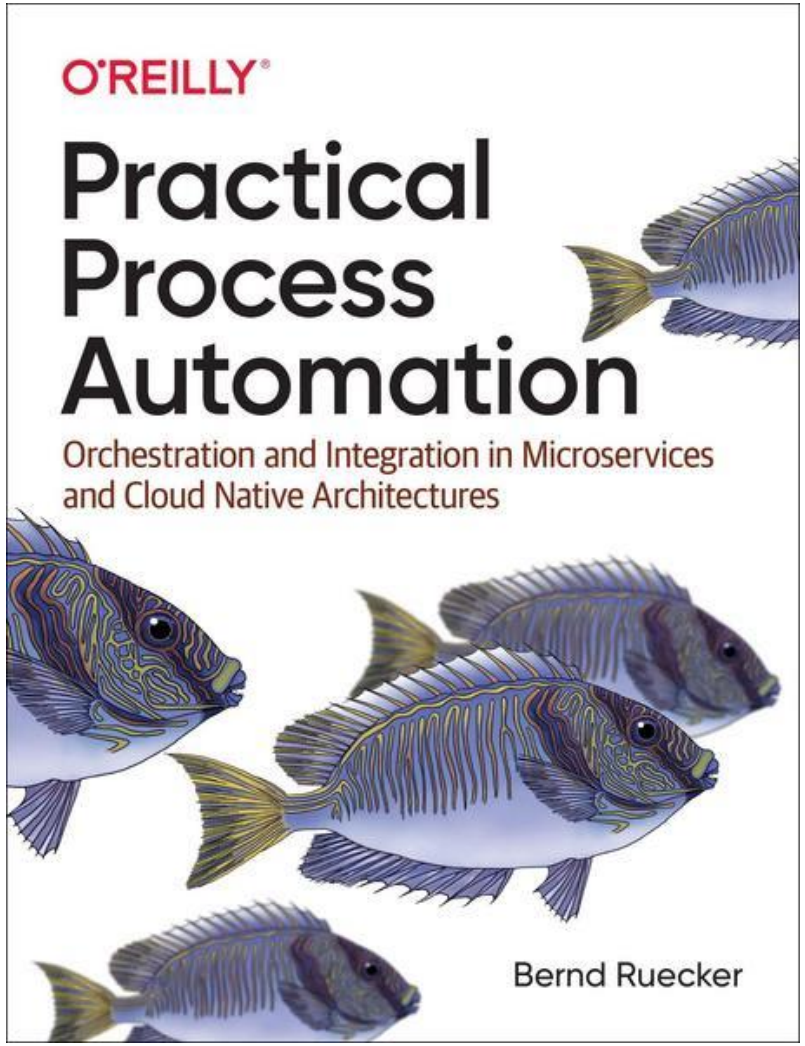


Testing and Versioning of Process Solutions

- From a technical point of perspective BPMN is a programming language. Thus, models should be treated like code
  - Blog article on testing process paths https://camunda.com/blog/2020/10/testing-entire-process-paths/

- Versioning of Process Solutions
  - If you deploy a changed process model, a **new version** is created
    - Active instances will continue on the old version
    - Newly created instances will run in the new version
  - Migration of active instances to new version
  - Versioning glue code and data definitions

Further References



Based on Chapter 3 – Developing Process Solutions

Agenda

- Workflow Engines: When and Why?
- Development of Process Solutions
- **Orchestrating Everything**

Process Engines for Orchestration

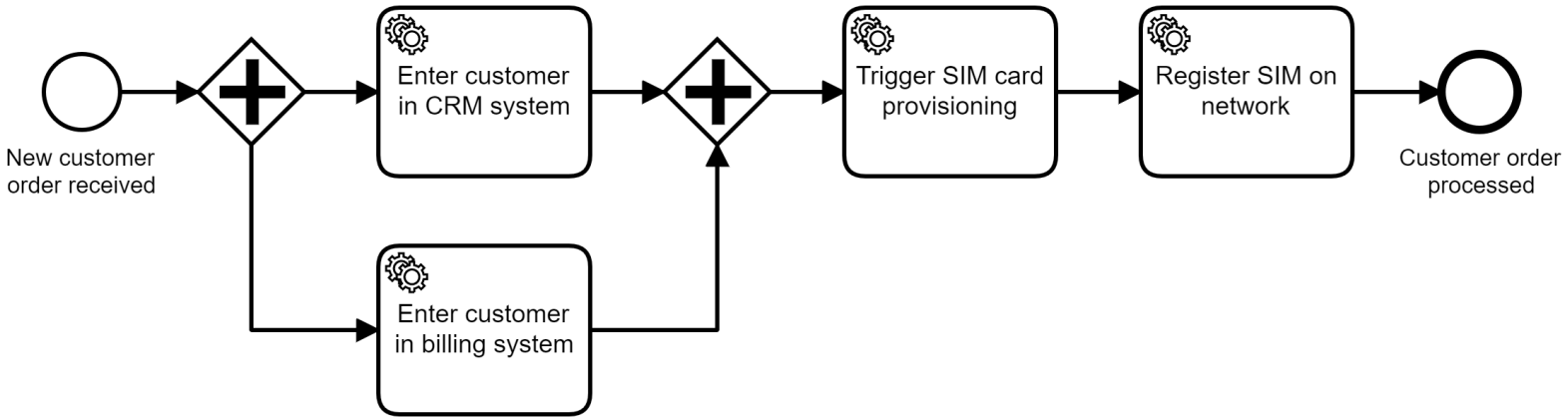
- Software components
- Decisions
- Humans
- RPA bots
- Physical devices and things
- AI agents

Process Engines for Orchestration

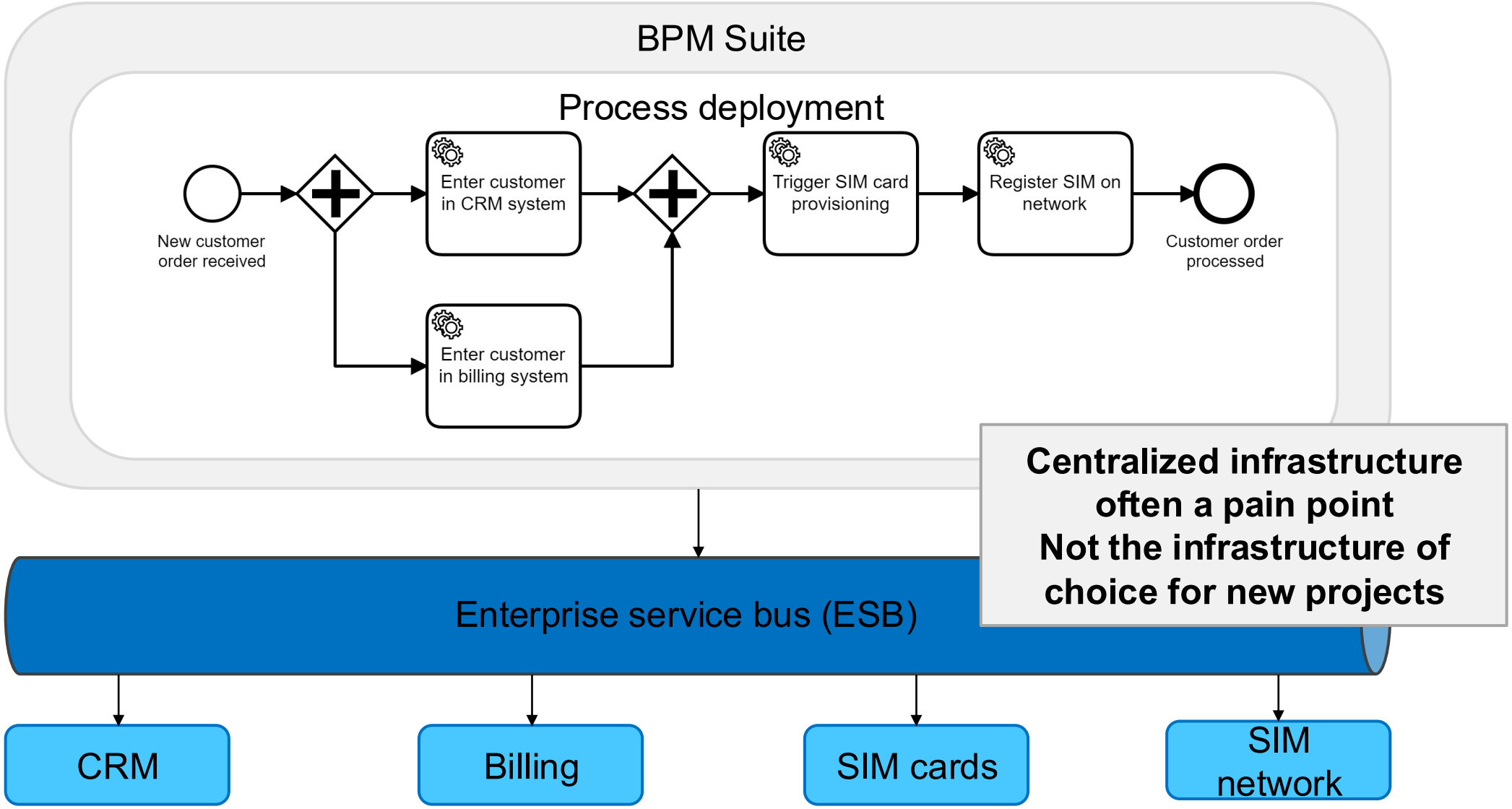
- **Software components**
- Decisions
- Humans
- RPA bots
- Physical devices and things
- AI agents

Process Engines for Orchestration
Orchestration of Software Components

- A workflow engine can orchestrate anything with an API
- Depending on the architectural style, the way you operate the process engine and design the model differs

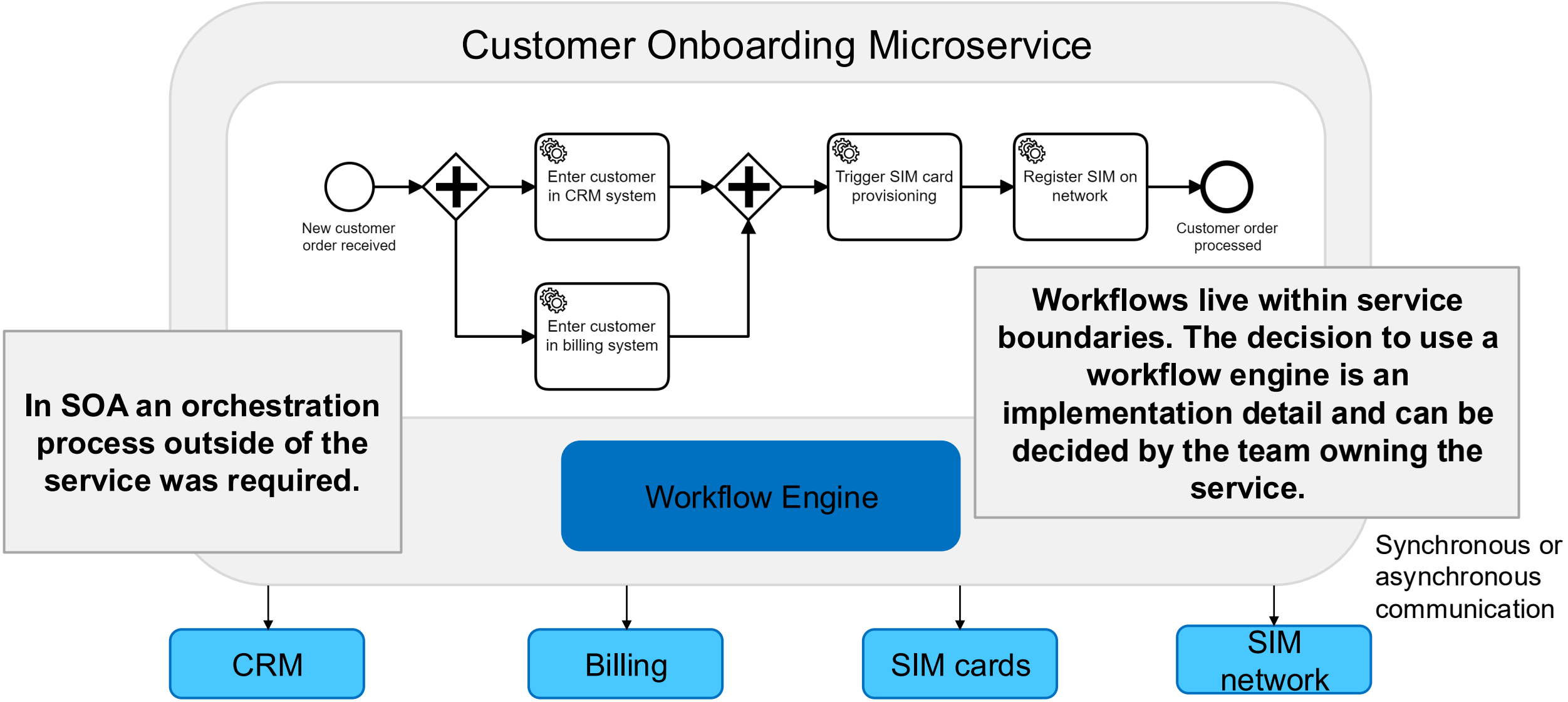


Orchestrating Software
Service-Oriented Architecture (SOA)



Centralized infrastructure often a pain point
Not the infrastructure of choice for new projects

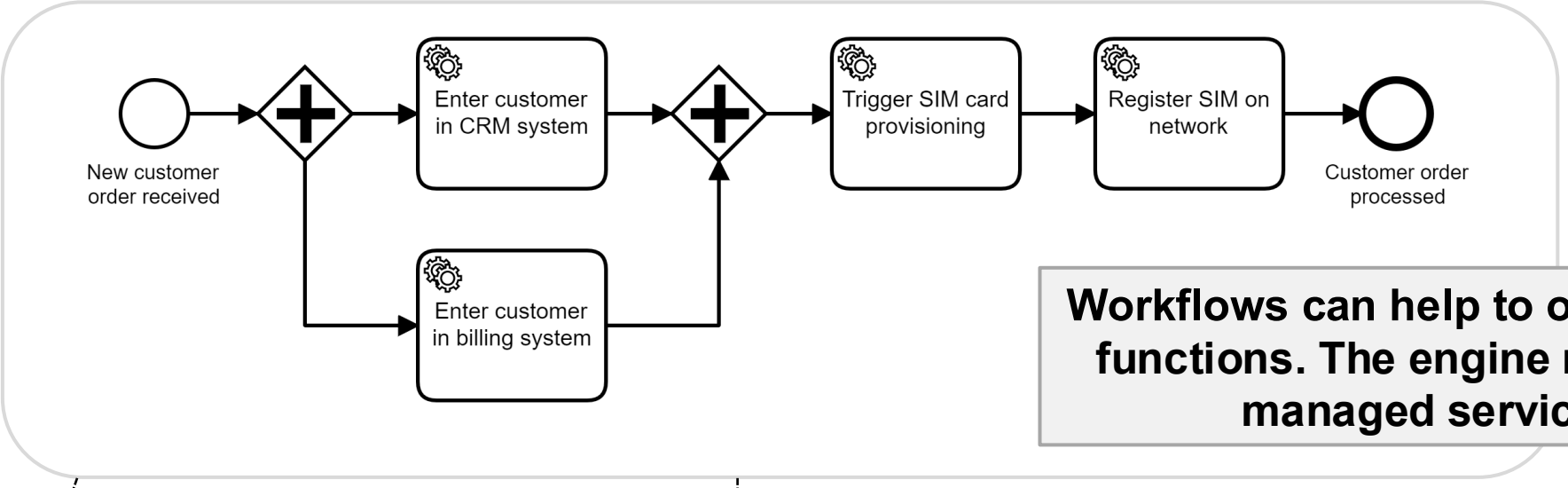
Orchestrating Software Microservices



In SOA an orchestration process outside of the service was required.

Workflows live within service boundaries. The decision to use a workflow engine is an implementation detail and can be decided by the team owning the service.

Orchestrating Software
Serverless Functions



Workflows can help to orchestrate functions. The engine runs as a managed service.

Owned by this team

Deployed on this managed engine

Function: onboard customer

Starts workflow

Workflow Engine

Invokes function

Function: create in CRM

Function: create in billing

Function: issue SIM card

Function: register SIM

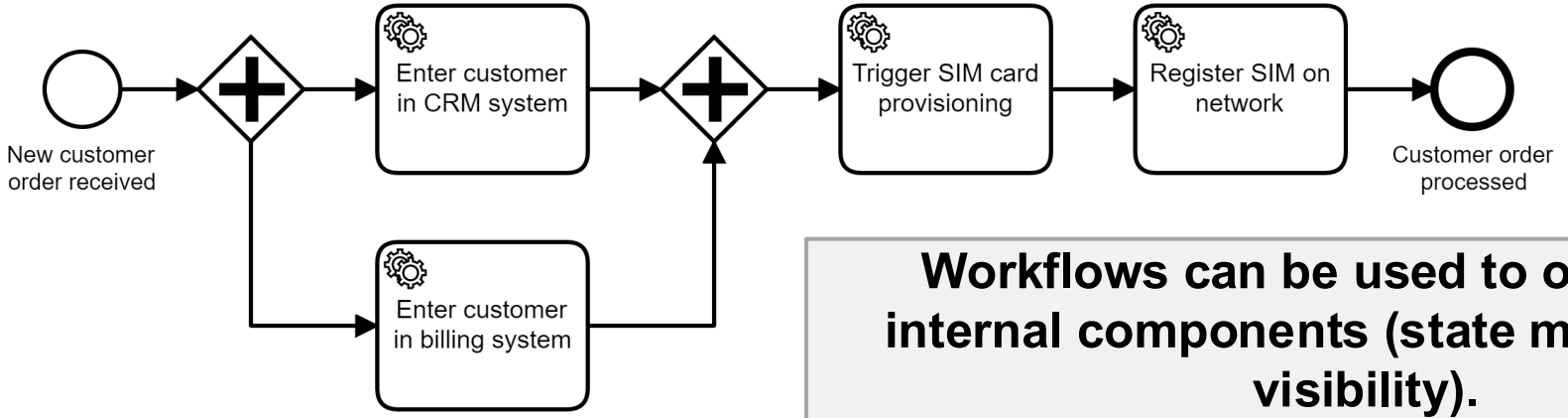
Orchestrating Software
Modular Monoliths

Monolith

Modules

CRM | Billing | SIM cards | SIM network | SIM cards | SIM network

Resources



Workflows can be used to orchestrate internal components (state management, visibility).
Changes to model can be potentially redeployed without deploying the whole monolith.

Workflow Engine

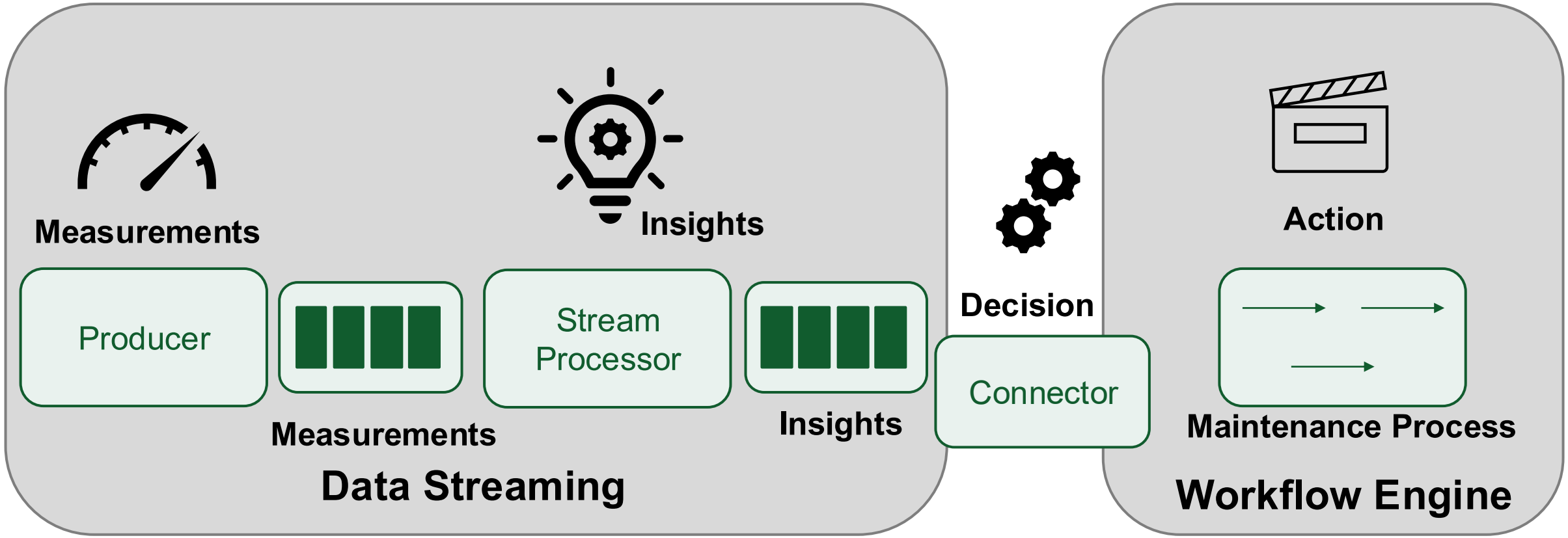
Libraries

Process Engines for Orchestration

- Software components
- Decisions
- Humans
- RPA bots
- **Physical devices and things**
- AI agents

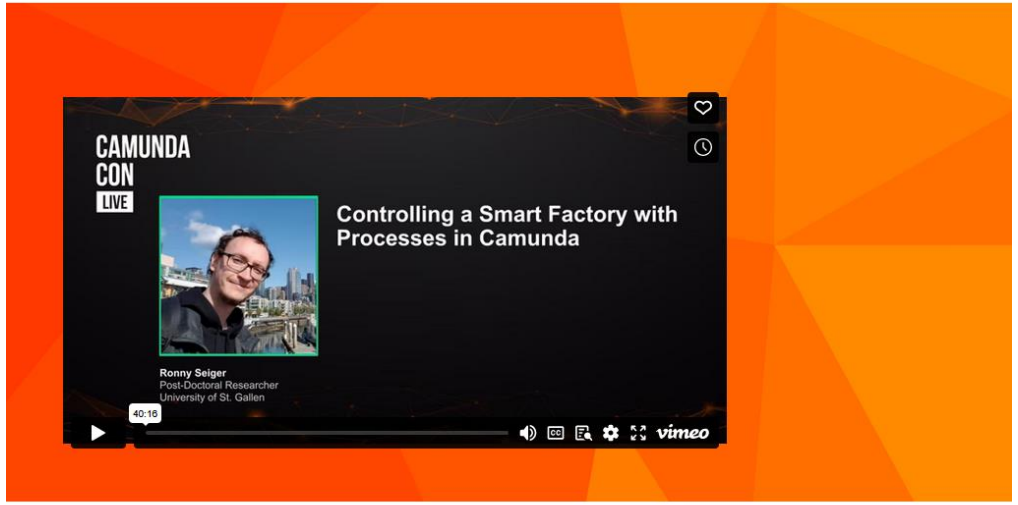
Process Engines for Orchestration
Orchestration of Pyhsical Devices and Things

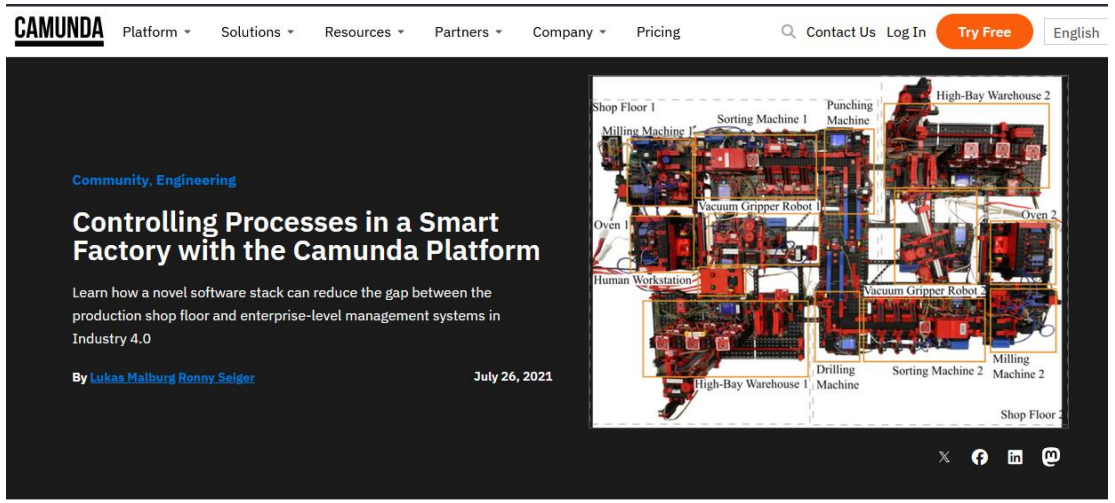
Focus after the break



Process Engines for Orchestration
Orchestration of Pyhsical Devices and Things

CAMUNDA



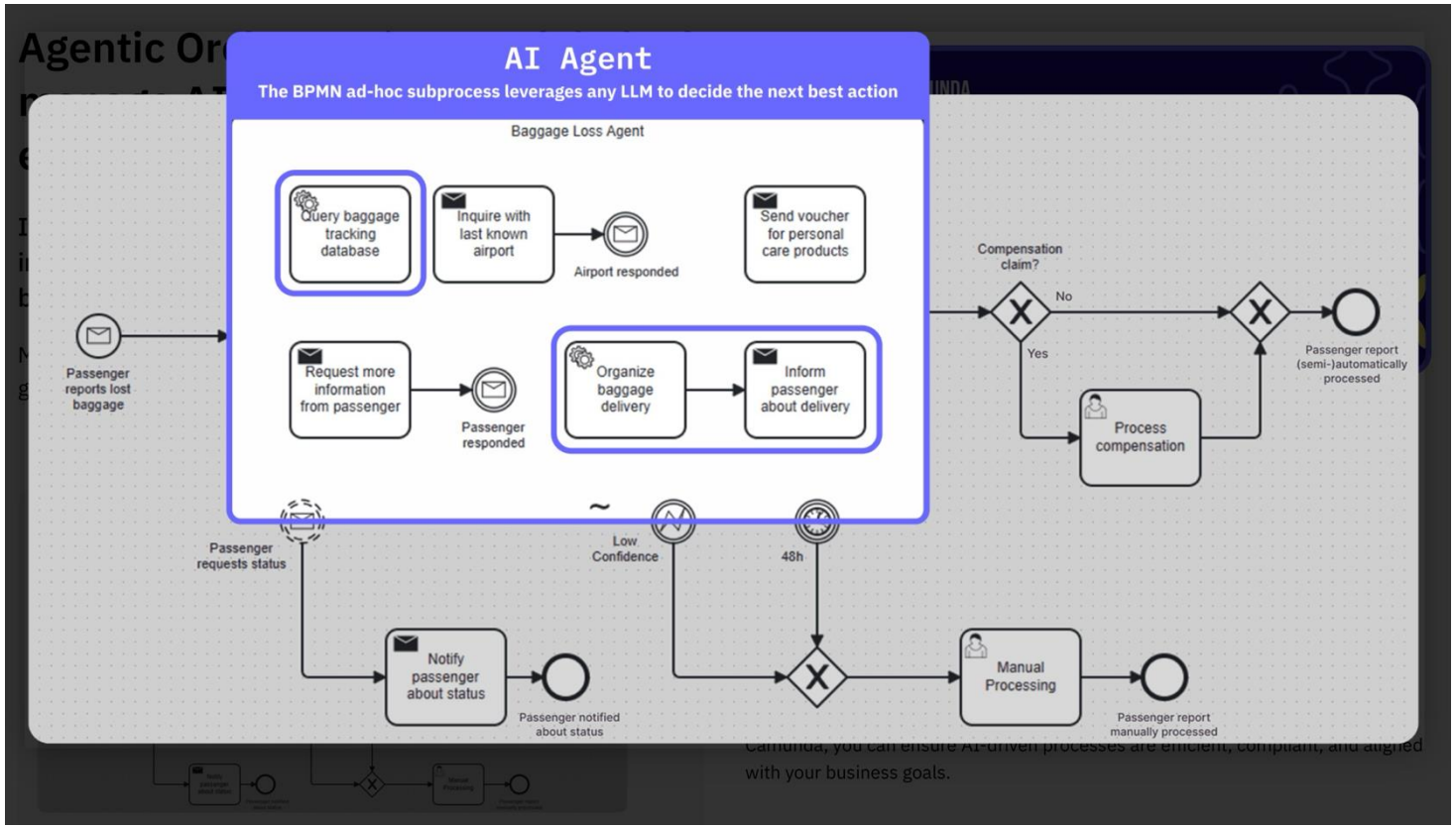


Blog > Controlling Processes in a Smart Factory with the Camunda Platform

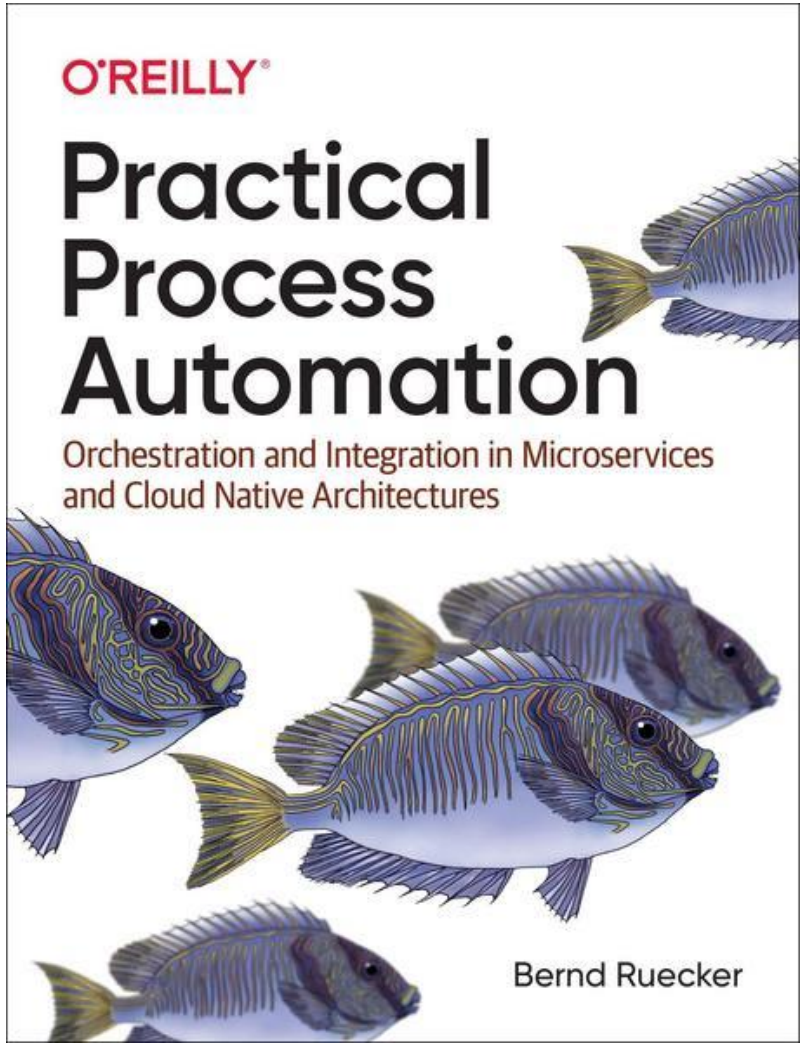
Process Engines for Orchestration

- Software components
- Decisions
- Humans
- RPA bots
- Physical devices and things
- **AI agents**

Process Engines for Orchestration
Orchestration of AI Agents



Further References



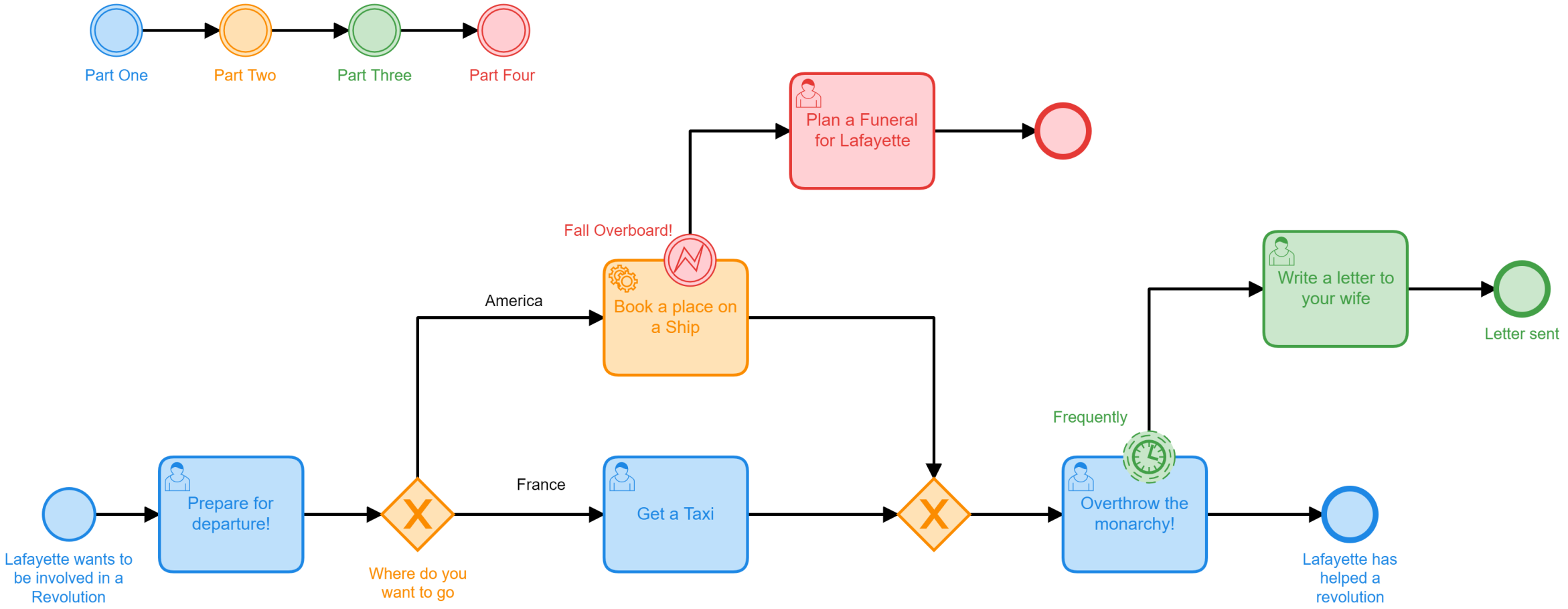
Based on Chapter 4 – Orchestrating Everything

Further References

- BPMN Standard: https://www.iso.org/standard/62652.html

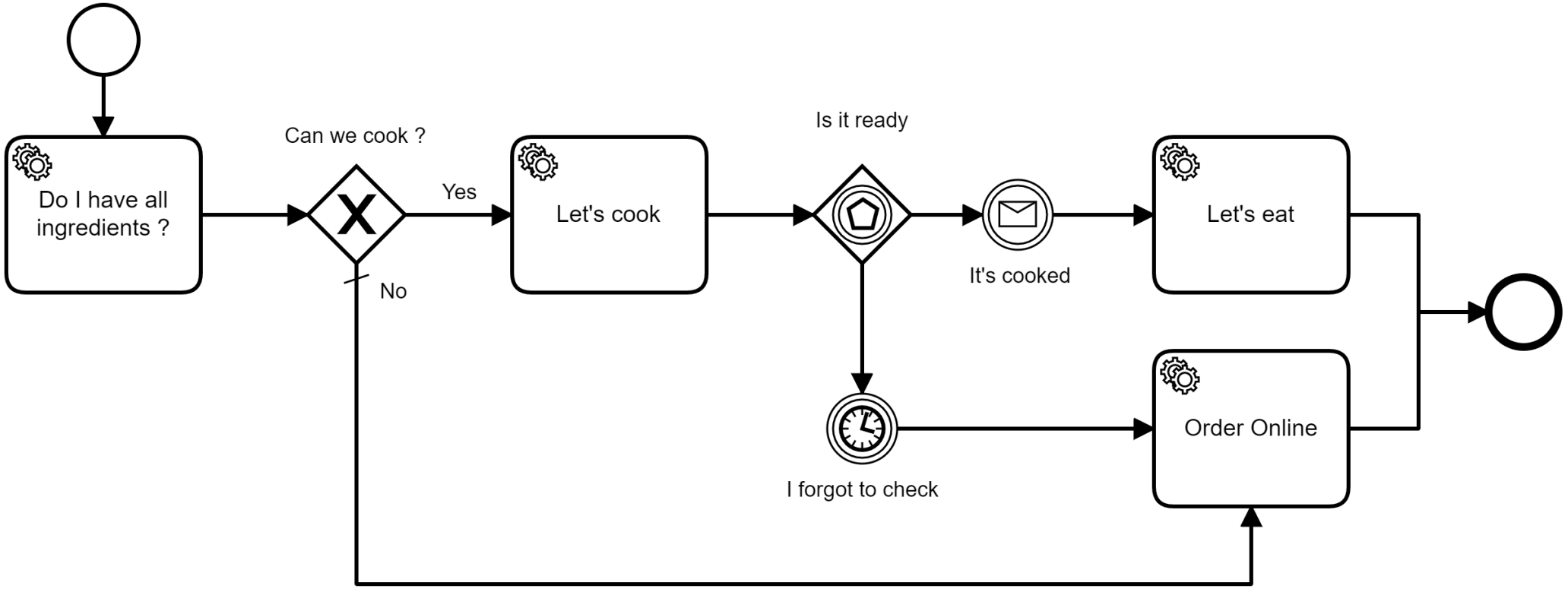
- Camunda Spring Boot Tutorial: https://github.com/camunda-community-hub/Camunda-Spring-Boot-Tutorial-Lafayette

Example Process Solution using Java and Spring Boot



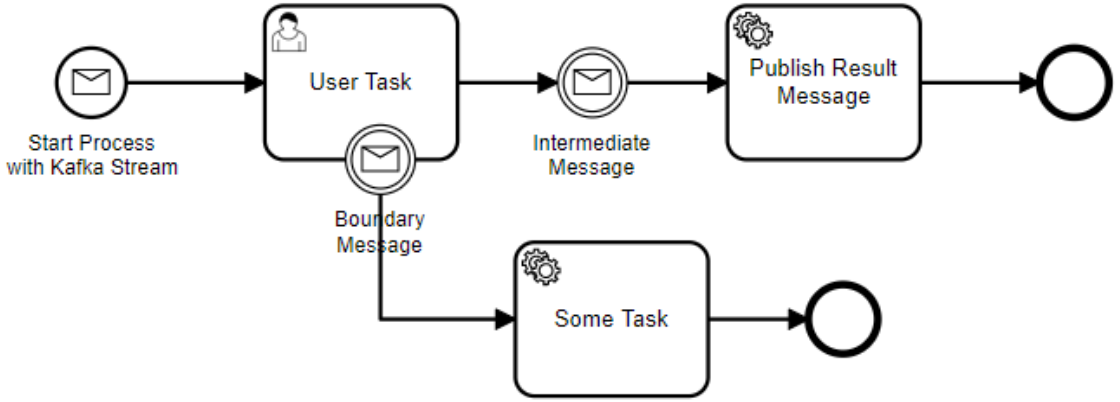
Camunda Spring Boot Tutorial: https://github.com/camunda-community-hub/Camunda-7-Spring-Boot-Tutorial-Lafayette

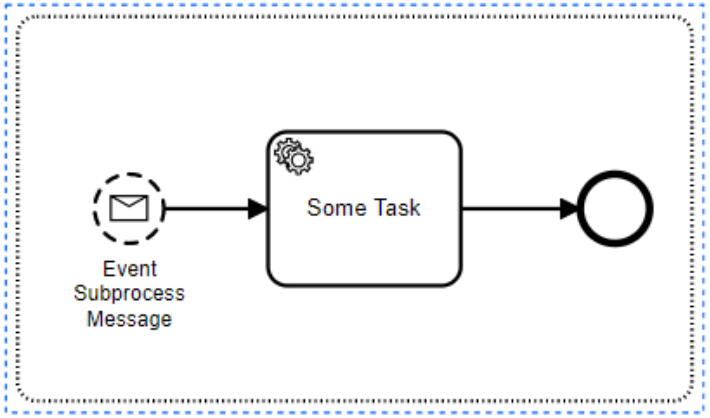
Example Process Solution using Java and Spring Boot



https://github.com/sourabhparsekar/camunda-masala-noodles
https://medium.com/nerd-for-tech/bpmn2-0-camunda-workflow-spring-boot-application-2381f3d42e5f

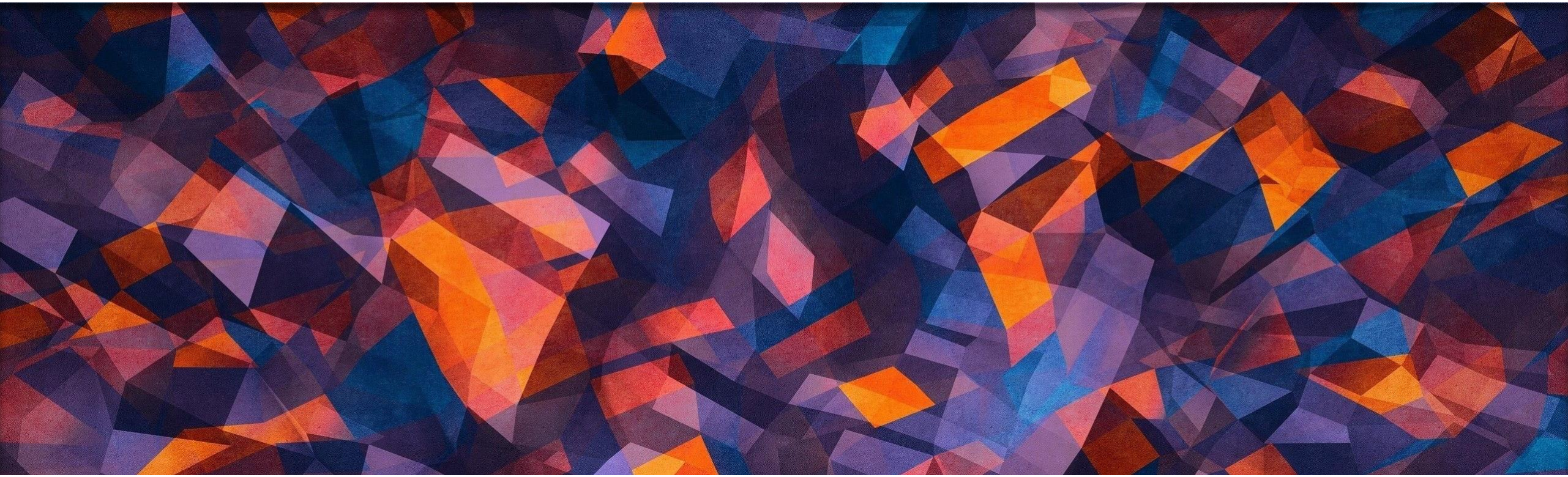
Example Process Solution with Message Events and Kafka as a Broker





https://github.com/scs-edpo/lab06-camunda-kafka-demo

Universität St.Gallen



Event-driven and Process-oriented Architectures

Exercise

Lecture 3

- Lab 5: Development of a Process Solution
  - Service Tasks using Java Delegates and External Task Worker, User Tasks, Gateways, HTML user interface for starting the process, Rest Controller
  - The Operaton fork of Camunda
  - https://github.com/scs-edpo/lab05-customer-onboarding-camunda
  - https://www.youtube.com/playlist?list=PLJG25HImvsOVssaiPmavxv3htN_dXS3BW

- Lab 6: Camunda Kafka Demo
  - Checkout and try to run the Camunda Kafka Demo
  - Execute the example process and investigate message exchange on Kafka
  - https://github.com/scs-edpo/lab06-camunda-kafka-demo

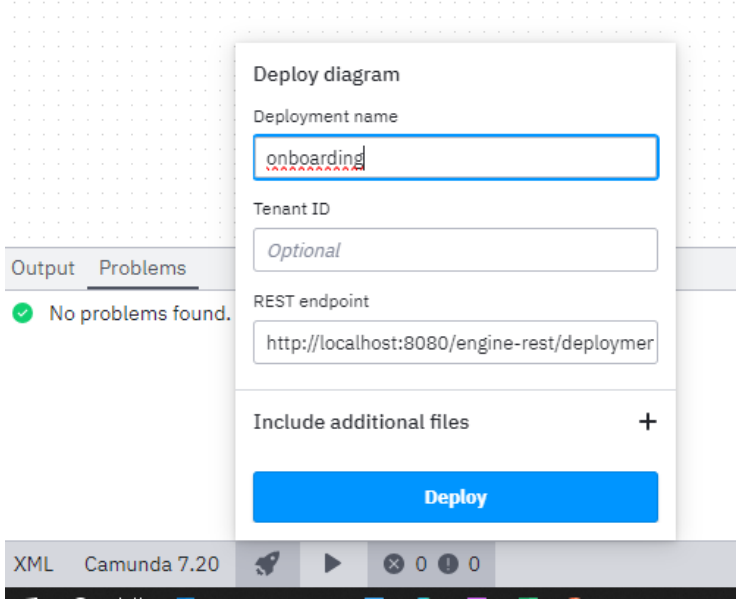
Camunda Modeler

- Download here
- Create new BPMN Diagram (Camunda 7)
- Model the process
- Deploy to Camunda/Operton Platform (Lab 5 or 6 must be running)

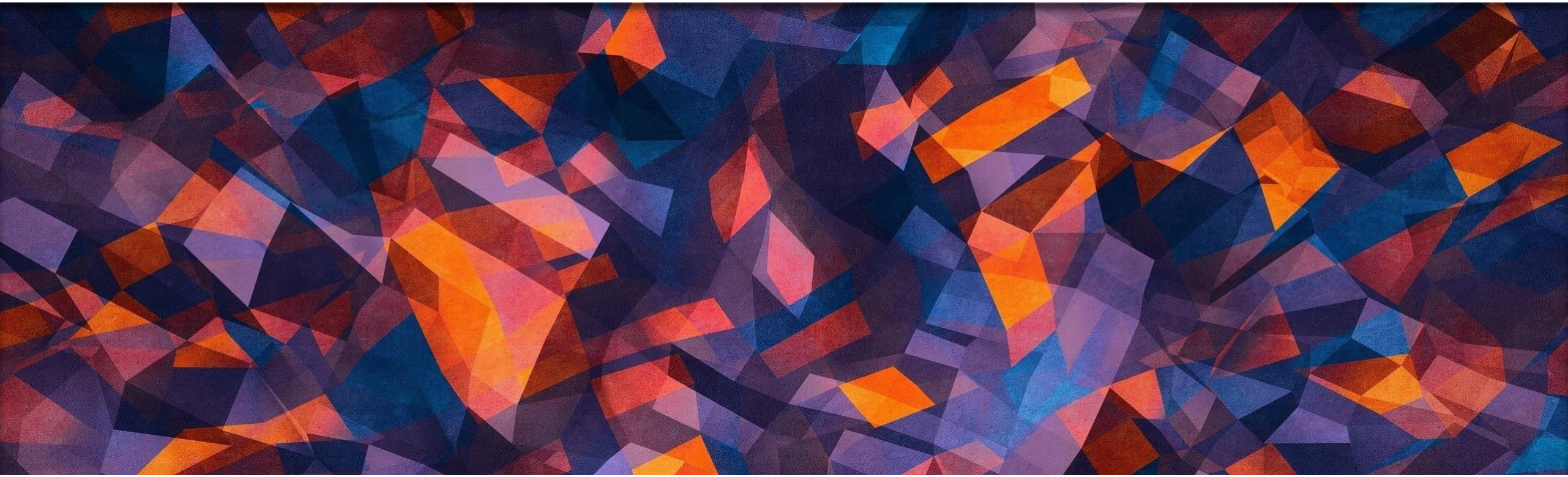
http://localhost:8080/engine-rest/deployment/create

- Web UI

http://localhost:8080 (demo / demo)



Universität St.Gallen



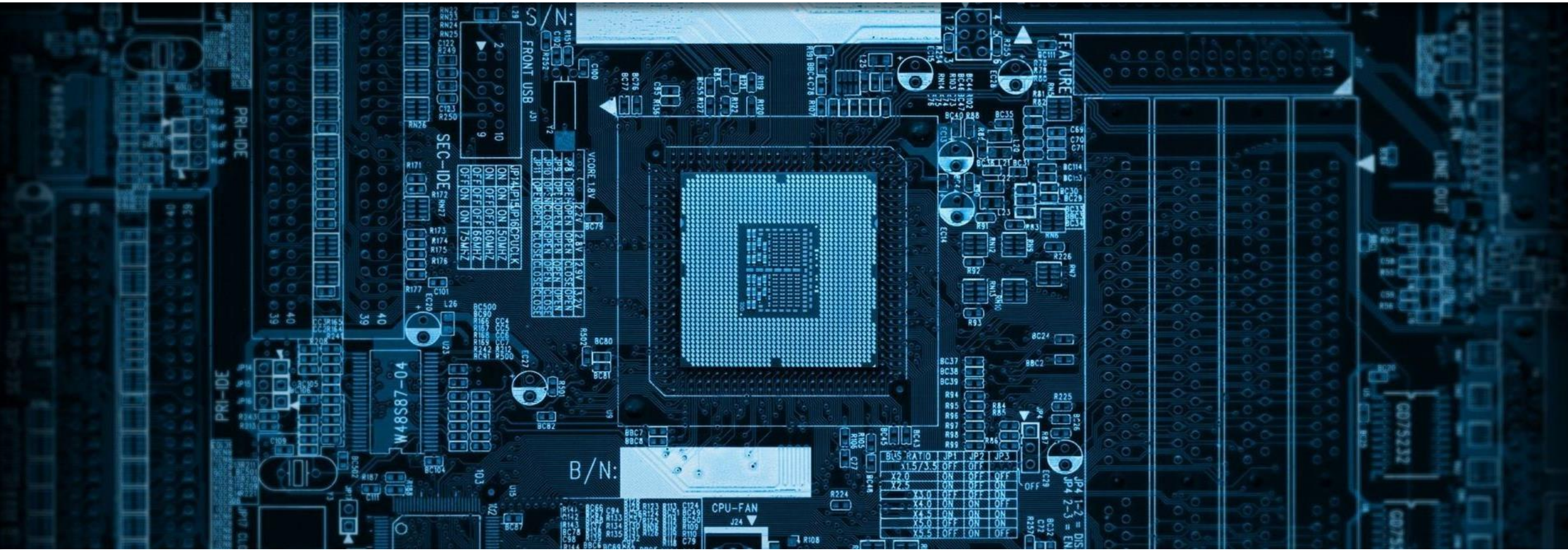
Event-driven and Process-oriented Architectures

Ideas for Project

Lecture 3

Ideas for Project

- Implement a process solution using the Spring Boot Framework and Camunda/Operaton
- Design (model) the process first
  - (Wait for the intro of Camunda 8 until next week)
- It is highly recommended to use Kafka as message broker.



Questions?